Modul Ajar

Mobile Programming



Disusun oleh:

Arif Hidayah, M.Tr.Kom.

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, modul pembelajaran *Mobile Programming* ini dapat disusun dan diselesaikan dengan baik. Modul ini disusun sebagai bahan ajar dalam kegiatan perkuliahan yang bertujuan untuk memberikan pemahaman praktis dan teoritis kepada mahasiswa mengenai pengembangan aplikasi mobile menggunakan Flutter sebagai framework utama.

Di dalam modul ini, pembahasan mencakup berbagai topik penting seperti dasar-dasar Flutter, penggunaan widget, manajemen state, integrasi database lokal dengan SQLite, pengelolaan media (gambar, video, audio), hingga penggunaan arsitektur modern dan manajemen navigasi yang efektif. Selain itu, modul ini juga membahas penggunaan GetX sebagai solusi manajemen state dan dependency injection yang efisien dan mudah digunakan.

Modul ini disusun secara bertahap mulai dari konsep dasar hingga implementasi studi kasus dan praktik langsung. Diharapkan mahasiswa dapat mengikuti materi secara runtut dan memperoleh keterampilan teknis dalam membangun aplikasi mobile lintas platform (Android dan iOS) yang fungsional dan interaktif.

Akhir kata, semoga modul ini dapat memberikan manfaat yang optimal bagi mahasiswa maupun pihak lain yang ingin mempelajari pengembangan aplikasi mobile dengan Flutter.

DAFTAR ISI

KATA PENGANTAR....2
DAFTAR ISI....3
DAFTAR GAMBAR....6
DAFTAR TABEL....7
CHAPTER 1 INTRODUCTION....8
1.1 Deskripsi Mata Kuliah....8
1.2 Capaian Pembelajaran....8
1.2.1 Capaian Pembelajaran Lulusan Program Studi (CPL-Prodi)....8
1.2.2 Capaian Pembelajaran Mata Kuliah (CPMK)....8
1.3 Media Pembelajaran....9
1.3.1 Perangkat Lunak....9
1.3.3 Pendukung....10
CHAPTER 2 PENGENALAN FLUTTER DAN DART....11
2.1 Kemampuan Akhir yang Diharapkan....11
2.2 Bahasa Pemrograman Android....11
2.3 Apa Itu Dart?....13
2.4 Platform Dart....14
2.3.1 Flutter....14
2.3.2 Web....15
2.3.3 Server....15
2.5 Instalasi VSCode....16
2.6 Instalasi Flutter....19
2.7 Pengaturan Device Untuk Debugging....22
2.8 Membuat Project di Flutter....25
CHAPTER 3 PENGENALAN FLUTTER DAN DART....26
3.1 Menyiapkan Emulator Android (Android Studio)....26
3.2 Install ekstensi pada Visual Studio Code....27
3.3 Membuat dan Menjalankan Project Flutter....28
3.4 Pengenalan Bahasa Dart....29
3.3.1 Karakteristik Dart....29
3.3.2 Tipe Data....30
3.3.3 Final vs Const....31
3.3.4 Comment....32
3.3.5 Function....33
3.3.6 Sync dan Async....34
3.5 Object Oriented Programming (OOP) Dart....47
3.5.1 Konsep OOP....47
3.5.2 Class & Object....48
3.5.3 Constructor....50
3.5.4 Inheritance....50
3.5.5 Getter & Setter....51
3.5.6 Abstract Class....52
3.6 Flutter Dasar....52
CHAPTER 4 MANAJEMEN LAYOUT DAN WIDGET PADA FLUTTER....55
4.1 Pengenalan Widget dalam Flutter....55
4.2 Widget Utama pada Flutter....56
4.2.1 Material App....56
4.2.2 Scaffold....57

4.2.3 AppBar 58
4.2.4 Container 58
4.3 Widget Dasar (Visible Widget) 60
4.3.1 Text 60
4.3.2 Flutter Logo 60
4.3.3 ElevatedButton 61
4.3.4 Icon 62
4.3.5 Image 63
4.4 Widget Layout (Invisible Widgets/Layouting) 64
4.4.1 Center 64
4.4.2 Column 64
4.4.3 Row 65
4.4.4 Stack 65
4.4.5 Single Child Scroll View 65
4.4.6 List View 66
4.4.7 Grid View 66
4.5 Ringkasan dan Kesimpulan 66
4.5.1 Perbedaan Visible vs Invisible Widget 66
4.5.2 Pentingnya Layouting dalam Flutter 67
4.5.3 Tips Mengelola Layout secara Efisien 67
CHAPTER 5 MANAJEMEN DAN NAVIGASI STATE 68
5.1 Stateless vs Stateful Widget 68
5.1.1 Stateless Widget 68
5.1.2 Stateful Widget 68
5.2 Komponen UI Interaktif 69
5.2.1 Dialog 69
5.2.2 SnackBar 70
5.2.3 TextField 70
5.2.4 DropDown 71
5.2.5 TabBar 72
5.3 Navigasi Antar Halaman 73
5.3.1 BottonNavigationBar 73
5.3.2 BottomSheet 74
5.3.3 Drawer 75
5.3.4 Navigator 76
5.4 Ringkasan dan Kesimpulan 76
CHAPTER 6 CRUD SQLITE 77
6.1 Konsep dasar Sqflite 77
6.2 Implementasi CRUD menggunakan Sqflite 77
6.2.2 Struktur Folder Project 78
6.2.3 Membuat Model Note 79
6.2.4 Membuat DBHelper 82
6.2.5 Membuat CRUD UI 88
6.2.6 Menjalankan main.dart 97
CHAPTER 7 PENGELOLAAN MEDIA DALAM APLIKASI 98
7.1 Pengelolaan Gambar 98
7.2 Pengelolaan Video 100
7.3 Pengelolaan Audio 103
CHAPTER 8 DEPLOYMENT FLUTTER 106
8.1 Pengantar Deployment 106
8.2 Persiapan Deployment 106
8.2.1 Instalasi dan Konfigurasi Lingkunga 106

8.2.2 Menyiapkan Project 107
8.2.3 Konfigurasi Splash Screen 107
8.3 Deployment Untuk Platform 108
8.3.1 Android (APK & AAB) 108
8.3.2 iOS 110
8.3.3 Web 110
8.3.4 Desktop 111
8.4 Tips Deployment 111
8.5 Ringkasan Perintah Penting 111

DAFTAR GAMBAR

Gambar 2. 1 Logo Dart 13
Gambar 2. 2 Download Visual Studio Code 17
Gambar 2. 3 Setup Installasi 17
Gambar 2. 4 Accept the Agreement 18
Gambar 2. 5 Additional Tasks 18
Gambar 2. 6 Install Finish 19
Gambar 2. 7 Plugin Flutter 19
Gambar 2. 8 Environment Variables 20
Gambar 2. 9 Variable Path 21
Gambar 2. 10 Add Variable 21
Gambar 2. 11 Flutter Doctor 22
Gambar 2. 12 Settings Android 22
Gambar 2. 13 Tentang Telepon 23
Gambar 2. 14 Setelan Tambahan 23
Gambar 2. 15 Opsi Pengembang 23
Gambar 2. 16 Aktifkan Opsi Pengembang 24
Gambar 2. 17 Debugging USB 24
Gambar 2. 18 Pop Up Peringatan 25
Gambar 3. 1 Virtual Device Manager 26
Gambar 3. 2 Tambah Device Baru 26
Gambar 3. 3 Pilih Device 26
Gambar 3. 4 Pilih API 27
Gambar 3. 5 Nama AVD 27
Gambar 3. 6 AVD Telah Ditambahkan 27
Gambar 3. 7 Install Ektensi 27
Gambar 3. 8 New Project 28
Gambar 3. 9 Pilih Application 28
Gambar 3. 10 Beri Nama Project 28
Gambar 3. 11 Flutter 29
Gambar 3. 12 Perbedaan Synchronous dan Asynchrounous 34
Gambar 3. 13 Ilustrasi OOP 48
Gambar 3. 14 Ilustrasi OOP 2 48
Gambar 4. 1 Papan Kayu 55
Gambar 4. 2 Kanvas 55
Gambar 4. 3 Row dan Column 56
Gambar 4. 4 Sumbu dan Layer Widget 56

DAFTAR TABEL

Tabel 1. 1 Perangkat Lunak.....9
Tabel 1. 2 Perangkat Keras 9
Tabel 1. 3 Pendukung.....10
Tabel 8. 1 Perbedaan APK dan AAB 108
Tabel 8. 2 Ringkasan Perintah.....112

CHAPTER 1
INTRODUCTION

1.1 Deskripsi Mata Kuliah

Mata kuliah Mobile Programming bertujuan untuk memberikan pengetahuan dan keterampilan kepada mahasiswa dalam mengembangkan aplikasi mobile modern berbasis Android menggunakan Flutter. Materi mencakup pengenalan Flutter dan Dart, manajemen layout dan widget, navigasi antar halaman, pengelolaan state, pemanfaatan media (gambar, audio, video), serta proses *build* dan deployment aplikasi. Mahasiswa akan mempelajari praktik pengembangan aplikasi yang user-friendly, interaktif, dan responsif sesuai dengan prinsip UI/UX yang baik.

1.2 Capaian Pembelajaran

1.2.1 Capaian Pembelajaran Lulusan Program Studi (CPL-Prodi)

Setelah mengikuti mata kuliah ini, mahasiswa diharapkan:

1. Mampu mengidentifikasi, merencanakan, merancang, mengevaluasi dan menerapkan solusi terhadap permasalahan pada bidang informatika (ilmu komputer)
2. Mampu mengkaji isu-isu di bidang informatika danmenunjukkan keahlian dalam merancang, menerapkan dan mengevaluasi baik rencana strategis maupun rencana pelaksanaan. Mampu menyusun ide, hasil pemikiran dan argumen saintifik untuk solusi permasalahan teknologi informatika
3. Mampu memilih dan menerapkan teknik dan perangkat yang paling sesuai (*best practice*) guna mendukung penyelesaian permasalahan teknologi informatika (ilmu komputer)
4. Mampu melibatkan diri dalam proses belajar terus menerus sepanjang hidup
5. Mampu bekerja-sama secara efektif dan menjadi pemimpin dalam bidang keprofesian teknologi informasi..

1.2.2 Capaian Pembelajaran Mata Kuliah (CPMK)

Setelah mengikuti mata kuliah ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dasar pemrograman mobile dan struktur proyek Flutter.
2. Mengembangkan antarmuka aplikasi menggunakan layout dan widget dalam Flutter.
3. Mengelola navigasi antar halaman dan state aplikasi.
4. Menyisipkan dan mengatur media (gambar, video, audio) dalam aplikasi.

5. Melakukan *build*, pengujian, dan *deployment* aplikasi ke perangkat Android.
6. Mendesain aplikasi mobile sesuai prinsip dasar UI/UX dan kebutuhan pengguna.

1.3 Media Pembelajaran

1.3.1 Perangkat Lunak

Tabel 1. 1 Perangkat Lunak

No.	Perangkat Lunak	Versi	Deskripsi/Fungsi
1	Flutter SDK	3.x+	Framework UI untuk membangun aplikasi mobile lintas platform (Android/iOS).
2	Android Studio	2022+	IDE resmi untuk pengembangan aplikasi Android dan debugging emulator.
3	Visual Studio Code	1.70+	Editor kode ringan dan fleksibel untuk Flutter/Dart development.
4	Dart SDK	3.x+	Bahasa pemrograman utama yang digunakan dalam Flutter.
5	Emulator Android	-	Alat untuk menjalankan dan menguji aplikasi tanpa perangkat fisik.
6	Node.js (opsional)	16+	Dibutuhkan saat mengintegrasikan backend sederhana (misal: Firebase CLI).
7	Firebase Tools (opsional)	-	Alat CLI untuk manajemen backend Firebase seperti auth, Firestore, hosting.

1.3.2 Perangkat Keras

Tabel 1. 2 Perangkat Keras

No.	Perangkat Keras	Versi	Deskripsi/Fungsi
1	Komputer/Laptop	-	Perangkat utama untuk menjalankan IDE, emulator, dan kompilasi aplikasi.
2	Smartphone Android	-	Untuk menguji langsung aplikasi pada perangkat fisik.

No.	Perangkat Keras	Versi	Deskripsi/Fungsi
3	LCD Projector	-	Digunakan untuk presentasi dan demo aplikasi di kelas.
4	USB Cable	-	Untuk menghubungkan perangkat Android ke laptop/PC saat debug fisik.

1.3.3 Pendukung

Tabel 1. 3 Pendukung

No.	Media Pendukung	Versi	Deskripsi/Fungsi
1	GitHub/GitLab	-	Platform version control dan kolaborasi kode antar mahasiswa.
2	Google Drive	-	Penyimpanan cloud untuk dokumentasi dan file pendukung perkuliahan.
3	Google Form & Spreadsheet	-	Digunakan untuk evaluasi, absensi, dan pengumpulan data tugas.
4	pub.dev (Flutter Packages)	-	Direktori resmi Flutter untuk mencari dan mengelola package/library tambahan.
5	Firebase Console	-	Backend platform (authentication, database, hosting) untuk aplikasi mobile.

CHAPTER 2
PENGENALAN FLUTTER DAN DART

2.1 Kemampuan Akhir yang Diharapkan

Setelah menyelesaikan materi ini, mahasiswa dapat mengetahui bahasa pemrograman Dart.

2.2 Bahasa Pemrograman Android

1. **Java**

   Java adalah bahasa pemrograman resmi pertama yang digunakan dalam pengembangan aplikasi Android sejak awal dirilisnya Android SDK oleh Google. Java merupakan bahasa yang berorientasi objek (OOP) dan memiliki komunitas pengembang yang sangat besar, serta dokumentasi yang lengkap.

   Kelebihan Java:

   - Kompatibilitas tinggi dengan Android SDK.
   - Tersedia banyak library dan framework pendukung.
   - Komunitas besar sehingga mudah mencari bantuan atau solusi.
   - Dokumentasi dan sumber belajar melimpah.

   Java cocok digunakan oleh pemula hingga profesional, dan masih digunakan secara luas dalam banyak proyek Android hingga saat ini.

2. **C++**

   C++ digunakan dalam pengembangan aplikasi Android yang memerlukan kinerja tinggi, seperti game atau aplikasi dengan pengolahan grafis dan multimedia. Bahasa ini tidak digunakan secara langsung, melainkan melalui NDK (Native Development Kit), yang memungkinkan bagian tertentu dari aplikasi ditulis dalam C++ untuk meningkatkan performa.

   Kelebihan C++:

   - Eksekusi sangat cepat.
   - Kontrol lebih besar terhadap sistem memori dan hardware.

   Namun, karena kompleksitas dan potensi terjadinya bug lebih tinggi, C++ umumnya digunakan oleh developer berpengalaman dalam proyek-proyek khusus.

3. Kotlin

Kotlin adalah bahasa pemrograman yang dikembangkan oleh JetBrains dan sejak 2017 telah diakui secara resmi oleh Google sebagai bahasa utama untuk pengembangan Android, mendampingi Java.

Keunggulan Kotlin:

- Sintaks lebih ringkas dan modern dibanding Java.
- Fitur null-safety bawaan yang mengurangi error.
- Dapat digunakan bersamaan dengan Java dalam satu proyek.
- Kompatibel penuh dengan Android Studio.

Kotlin kini banyak dipilih karena efisiensi pengembangan dan keterbacaannya yang tinggi.

4. Dart

Dart adalah bahasa pemrograman yang dikembangkan oleh Google dan digunakan untuk mengembangkan aplikasi multiplatform menggunakan framework Flutter. Dengan Dart, developer dapat membuat aplikasi untuk Android dan iOS hanya dari satu basis kode (*single codebase*), menghemat waktu dan sumber daya.

Kelebihan Dart:

- Kompilasi cepat dengan hot reload saat pengembangan.
- Sangat cocok untuk UI interaktif dan dinamis.
- Sintaks yang mudah dipelajari untuk pemula.

Flutter dan Dart menjadi pilihan favorit saat ini untuk pengembangan aplikasi mobile secara efisien.

5. C#

C# digunakan dalam pengembangan aplikasi Android melalui framework Xamarin, yang merupakan bagian dari platform .NET. Xamarin memungkinkan penggunaan kode C# untuk membangun aplikasi mobile cross-platform.

Kelebihan C# dengan Xamarin:

- Integrasi dengan Visual Studio yang kuat.
- Dapat membangun aplikasi Android dan iOS sekaligus.
- Ekosistem .NET yang luas.

C# biasanya digunakan oleh developer .NET yang ingin memperluas kemampuan mereka ke platform mobile.

6. JavaScript

JavaScript dapat digunakan untuk membangun aplikasi Android melalui framework seperti React Native dan Ionic. Framework ini memungkinkan pengembangan aplikasi hybrid dengan performa mendekati native.

Kelebihan JavaScript (React Native/Ionic):

- Mudah diakses oleh developer web.
- Proses development cepat dan efisien.
- Komunitas aktif dan banyak plugin siap pakai.

JavaScript adalah solusi tepat bagi yang berasal dari latar belakang web development.

7. TypeScript

TypeScript adalah bahasa pemrograman yang merupakan superset dari JavaScript. Bahasa ini menambahkan fitur pengetikan statis dan pengembangan skala besar ke dalam JavaScript.

Keunggulan TypeScript:

- Deteksi error saat compile time.
- Cocok untuk proyek besar dan kolaboratif.
- Digunakan pada framework seperti Angular, serta bisa dengan React Native.

Dengan TypeScript, pengembangan aplikasi menjadi lebih aman dan terstruktur.

2.3 Apa Itu Dart?



Gambar 2. 1 Logo Dart

Dart adalah bahasa pemrograman modern yang dikembangkan oleh Google dan dirancang untuk kebutuhan pengembangan aplikasi secara umum (*general purpose*). Dart mengedepankan produktivitas pengembang melalui sintaks yang sederhana, performa tinggi, serta dukungan terhadap paradigma pemrograman berorientasi objek (*object-oriented*).

Awalnya dikembangkan untuk menggantikan JavaScript dalam pengembangan web, Dart kini menjadi bahasa utama dalam membangun aplikasi multiplatform melalui framework Flutter. Kemampuannya dalam mengkompilasi ke native code membuatnya efisien digunakan di berbagai platform—baik mobile, web, maupun desktop.

Beberapa aplikasi besar yang menggunakan Dart antara lain:

- Google Adsense
- Google AdWords
- Google Fiber
- Aplikasi Hamilton (Android & iOS)

2.4 Platform Dart

2.3.1 Flutter

Flutter merupakan framework open-source yang dikembangkan oleh Google dan menggunakan bahasa Dart sebagai fondasinya. Framework ini dirancang khusus untuk membangun antarmuka pengguna (*User Interface/ UI*) yang modern, menarik, dan responsif. Dengan Flutter, pengembang dapat membuat aplikasi Android dan iOS hanya dengan satu basis kode, sehingga menghemat waktu dan tenaga dalam proses pengembangan lintas platform.

Salah satu kekuatan utama Flutter terletak pada sistem widget-nya yang fleksibel. Flutter menyediakan ratusan widget siap pakai dan mudah dikustomisasi untuk membangun elemen UI seperti tombol, teks, form, hingga layout kompleks. Selain itu, fitur hot reload pada Flutter memungkinkan pengembang untuk langsung melihat hasil perubahan kode secara real-time tanpa harus menjalankan ulang aplikasi, meningkatkan efisiensi dalam proses debugging dan iterasi desain.

Flutter juga mendukung berbagai plugin dan integrasi dengan pustaka pihak ketiga untuk menambahkan fitur seperti kamera, geolokasi, pembayaran, serta koneksi ke API atau database. Karena Flutter bersifat native compiled, performanya sangat optimal bahkan untuk aplikasi yang memerlukan animasi kompleks atau interaksi tinggi.

Dengan semua keunggulan tersebut, Flutter menjadi salah satu pilihan utama bagi pengembang mobile modern yang ingin membangun aplikasi yang tidak hanya berjalan di banyak platform, tetapi juga memiliki tampilan profesional dan performa tinggi.

2.3.2 Web

Selain digunakan dalam pengembangan aplikasi mobile melalui Flutter, Dart juga dapat digunakan untuk membangun aplikasi web, khususnya pada bagian front-end. Dart memiliki kemampuan untuk dikompilasi (atau lebih tepatnya *transpile*) menjadi JavaScript, sehingga hasil akhirnya tetap dapat dijalankan di seluruh web browser modern. Ini menjadikan Dart sebagai alternatif bahasa front-end yang efisien dengan sintaks modern dan dukungan strong typing.

Dalam konteks pengembangan web, Dart biasanya digunakan bersama beberapa framework populer. Salah satu yang paling dikenal adalah AngularDart, yaitu implementasi Angular yang diadaptasi untuk bekerja langsung dengan bahasa Dart. AngularDart cocok digunakan untuk membangun aplikasi web berbasis komponen dengan arsitektur yang terstruktur dan scalable. Selain itu, Dart juga dapat digunakan bersama Material Design Lite (MDL) for Dart, yang menyediakan kumpulan komponen UI modern berbasis prinsip desain material dari Google. MDL for Dart memungkinkan pengembang untuk membangun antarmuka yang konsisten dan responsif.

Framework lain yang juga digunakan adalah OverReact, yaitu framework Dart yang memanfaatkan konsep React-style UI namun tetap ditulis dalam bahasa Dart. OverReact digunakan untuk membuat komponen UI yang dinamis dan bisa digunakan ulang (reusable), memberikan fleksibilitas dalam pengembangan front-end berbasis Dart. Dengan ekosistem ini, Dart tetap menjadi bahasa yang relevan untuk pengembangan aplikasi web modern, khususnya bagi pengembang yang menginginkan konsistensi bahasa dalam membangun aplikasi multiplatform (mobile dan web) dari satu basis teknologi yang sama.

2.3.3 Server

Selain digunakan dalam pengembangan aplikasi mobile melalui Flutter, Dart juga dapat digunakan untuk membangun aplikasi web, khususnya pada bagian front-end. Dart memiliki kemampuan untuk dikompilasi (atau lebih tepatnya *transpile*) menjadi JavaScript, sehingga hasil akhirnya tetap dapat dijalankan di seluruh web browser modern. Ini menjadikan Dart sebagai alternatif bahasa front-end yang efisien dengan sintaks modern dan dukungan strong typing.

Dalam konteks pengembangan web, Dart biasanya digunakan bersama beberapa framework populer. Salah satu yang paling dikenal adalah AngularDart, yaitu implementasi Angular yang diadaptasi untuk bekerja langsung dengan bahasa Dart. AngularDart cocok

digunakan untuk membangun aplikasi web berbasis komponen dengan arsitektur yang terstruktur dan scalable. Selain itu, Dart juga dapat digunakan bersama Material Design Lite (MDL) for Dart, yang menyediakan kumpulan komponen UI modern berbasis prinsip desain material dari Google. MDL for Dart memungkinkan pengembang untuk membangun antarmuka yang konsisten dan responsif.

Framework lain yang juga digunakan adalah OverReact, yaitu framework Dart yang memanfaatkan konsep React-style UI namun tetap ditulis dalam bahasa Dart. OverReact digunakan untuk membuat komponen UI yang dinamis dan bisa digunakan ulang (reusable), memberikan fleksibilitas dalam pengembangan front-end berbasis Dart.

Dengan ekosistem ini, Dart tetap menjadi bahasa yang relevan untuk pengembangan aplikasi web modern, khususnya bagi pengembang yang menginginkan konsistensi bahasa dalam membangun aplikasi multiplatform (mobile dan web) dari satu basis teknologi yang sama.

2.5 Instalasi VSCode

Spesifikasi – spesifikasi nya yaitu :

a. Hardware, Perangkat keras rekomendasi yang dibutuhkan yaitu memiliki processor 1.6 GHz or faster processor, dan minimal memiliki 1 GB RAM.

b. Sistem operasi, Sistem operasi di windows minimal kalau memakai windows 7 wajib Menginstal .NET Framework 4.5.2 untuk sistem operasi lain yaitu :

    a. OS X Yosemite

    b. Windows 7, 8.0, 8.1 and 10 (32-bit and 64-bit)

    c. Linux (Debian): Ubuntu Desktop 14.04, Debian 7

    d. Linux (Red Hat): Red Hat Enterprise Linux 7, CentOS 7, Fedora 23

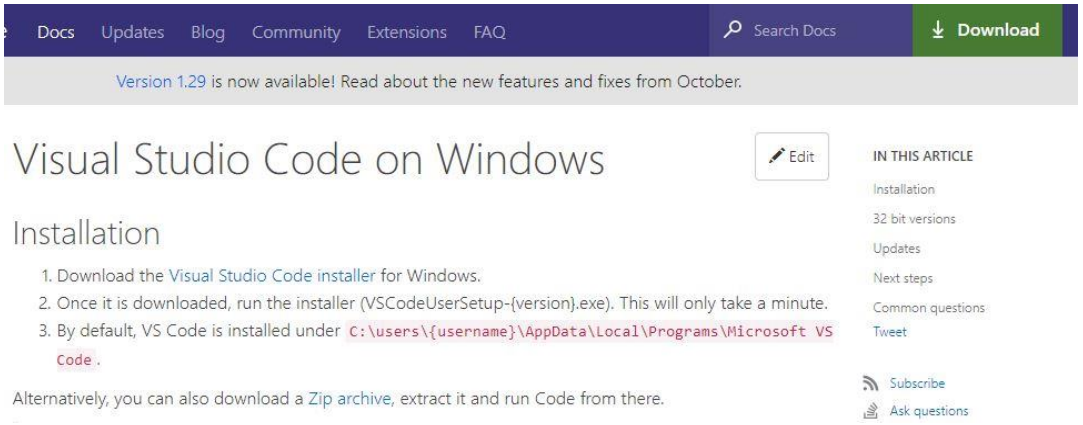
c. Persyaratan tambahan Linux yaitu:

    a. GLIBCXX version 3.4.15 or later

    b. GLIBC version 2.15 or later

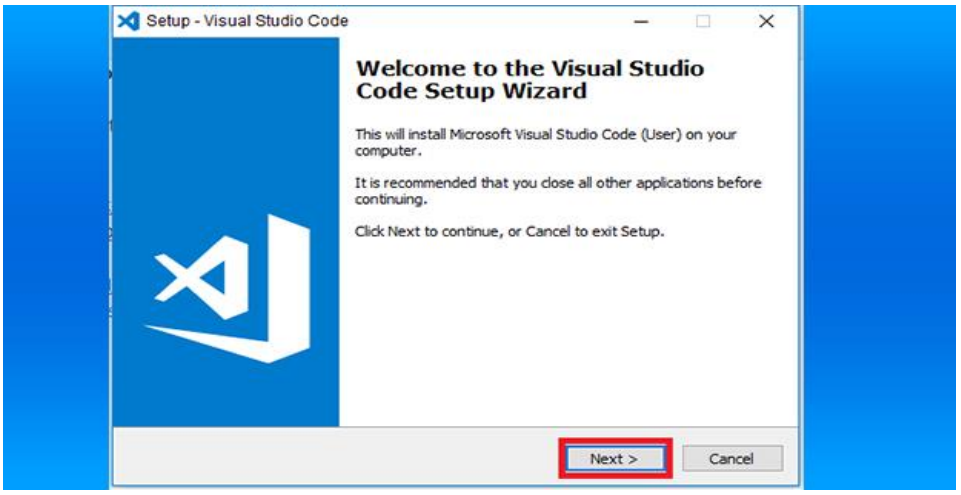
Setelah memastikan perangkat keras dan sistem operasi telah memenuhi spesifikasi minimum yang dibutuhkan, langkah selanjutnya adalah melakukan instalasi Visual Studio Code (VS Code). VS Code merupakan editor kode sumber yang ringan, cepat, dan mendukung berbagai bahasa pemrograman, termasuk Dart dan Flutter. Editor ini juga dilengkapi dengan berbagai ekstensi yang mempermudah proses pengembangan aplikasi mobile dan web.

a. Jika sudah di buka silahkan klik download di pojok kanan atas atau juga bisa dengan mengklik *"Download the Visual Studio Code installer for Windows."* seperti gambar di bawah ini.



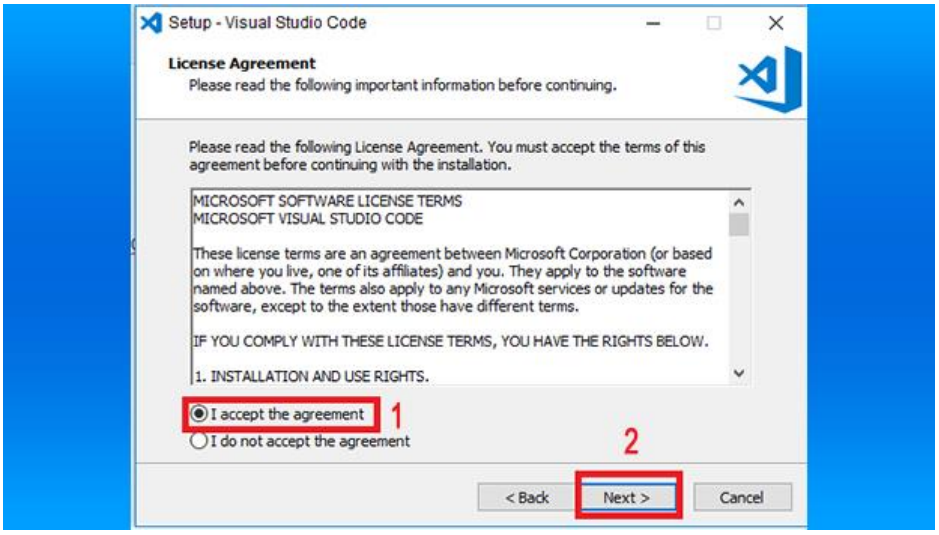
Gambar 2. 2 Download Visual Studio Code

b. Jika sudah klik/buka Aplikasi VSCode yang sudah di download, Kemudian ikuti kotak merah pada gambar klik *Next.*



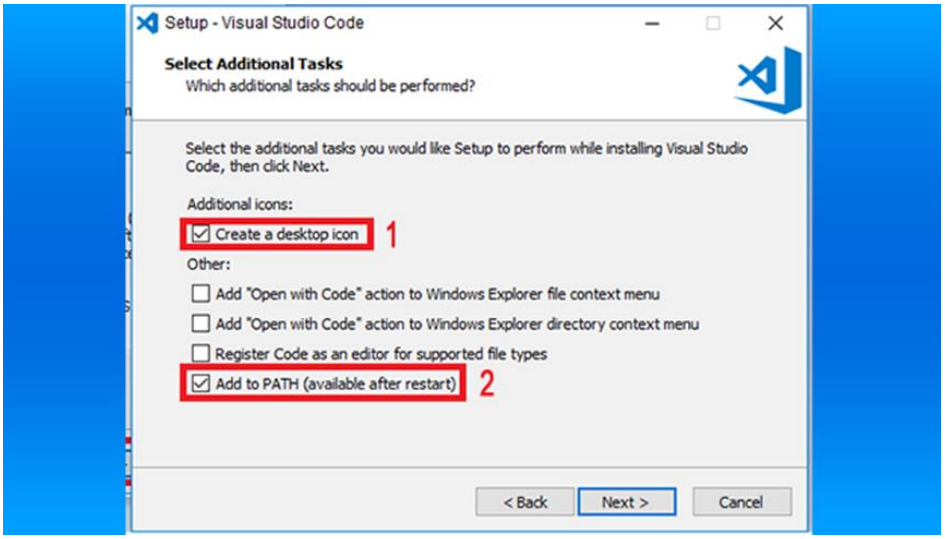
Gambar 2. 3 Setup Installasi

c. Kemudian pilih *"I accept the agreement"* untuk menyetujui kebijakan dari VSCode lalu klik *Next.*



Gambar 2. 4 Accept the Agreement

d. Kemudian klik Next lagi itu hanya penempatan direktori file programnya, lalu Next untuk membuat folder nya, Kemudian Ceklis "Create Desktop Icon" jika ingin membuat shortcut VSCode nya kemudian ceklis "Add to PATH (available after restart) jika sudah klik Next.



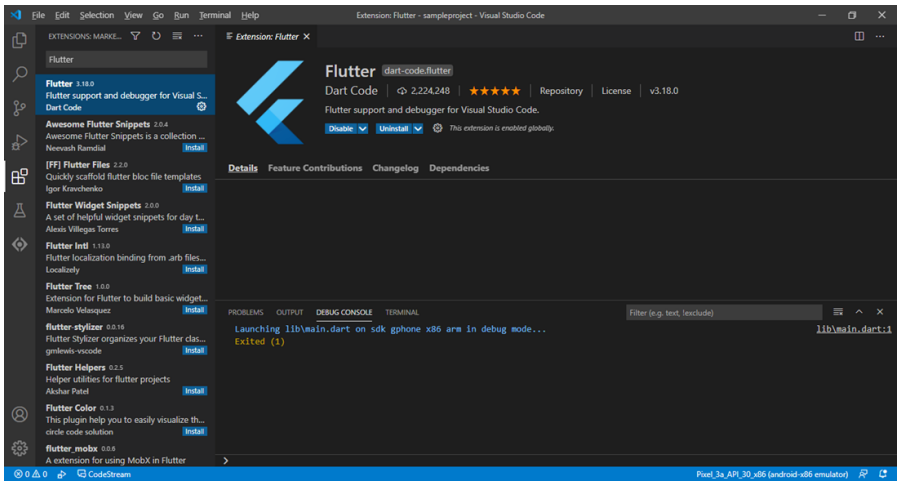
Gambar 2. 5 Additional Tasks

Kemudian Klik *Install* tunggu sampai *"Setup has finished installing Visual Studio Code on your komputer"* . kemudian klik *Finish* seperti gambar dibawah ini.



Gambar 2. 6 Install Finish

e. Install Plugin Flutter dan Dart



Gambar 2. 7 Plugin Flutter

2.6 Instalasi Flutter

a. Unduh Flutter versi stable disini

b. Setelah proses unduhan selesai, ekstrak file ZIP ke dalam folder pilihan. Catatan penting: Hindari penggunaan nama folder yang mengandung spasi. Contoh direktori:

   E:\src\flutter

c. Bagi sistem yang telah terinstal Git, proses pengunduhan juga dapat dilakukan dengan perintah terminal berikut:

```
git clone https://github.com/flutter/flutter.git -b stable
```

   Perintah ini akan mengunduh Flutter versi stabil langsung dari repositori resminya di GitHub.

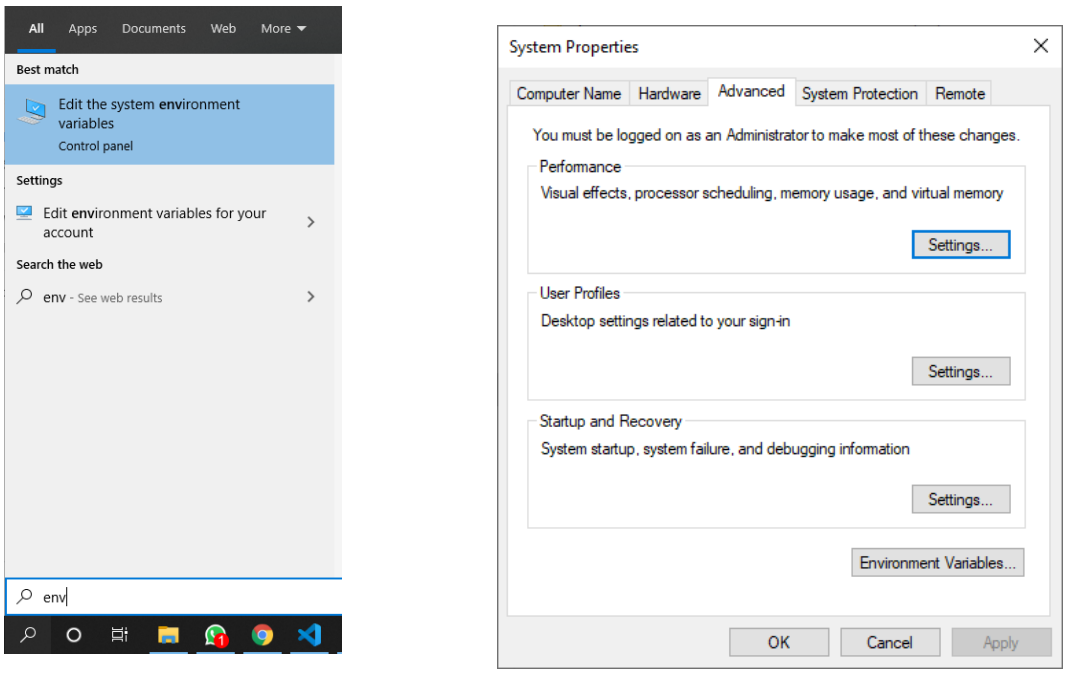
d. Konfigurasi Path

Agar perintah flutter dapat dikenali dan dijalankan melalui Command Prompt (CMD) atau PowerShell, langkah berikut perlu dilakukan untuk menambahkan Flutter ke dalam Environment Variables:

Langkah-langkah:

1. Buka System Environment

   Klik Edit the system environment variables, kemudian tekan Enter.



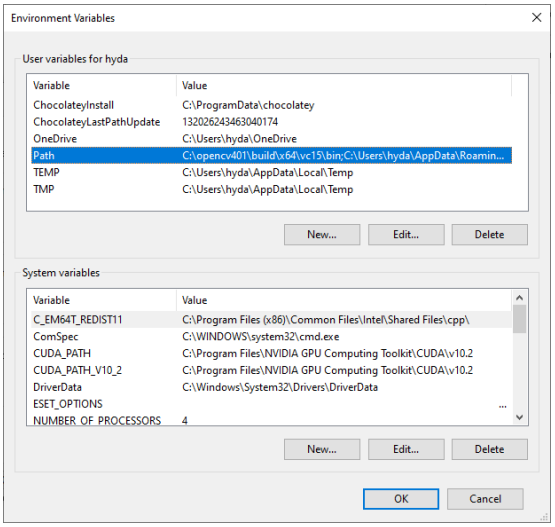
Gambar 2. 8 Environment Variables

2. Masuk ke Environment Variables

   Di jendela System Properties, klik tombol Environment Variables...

3. Pilih variabel PATH

   Di bagian User variables (atau System variables jika ingin global), pilih variabel Path, lalu klik tombol Edit...



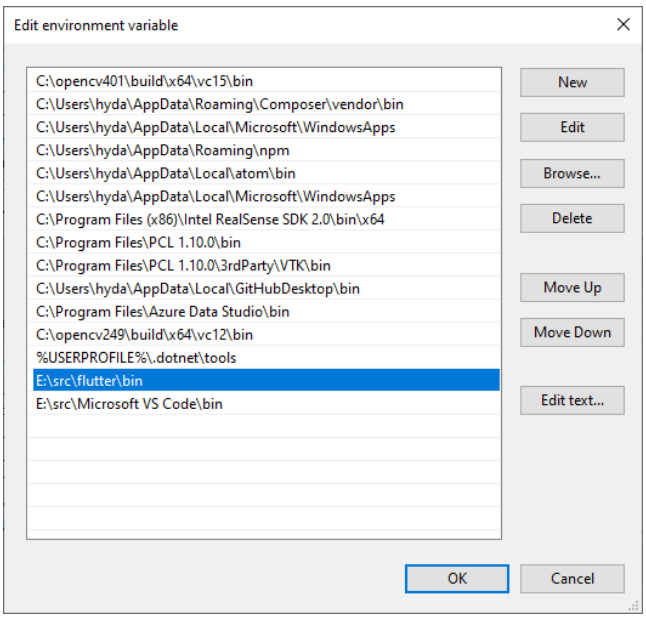
Gambar 2. 9 Variable Path

4. Tambahkan direktori Flutter

   Di jendela baru, klik tombol New, lalu masukkan direktori bin dari folder Flutter. Contoh:

   E:\src\flutter\bin

   Setelah selesai menambahkan, klik OK pada setiap jendela hingga tertutup.



Gambar 2. 10 Add Variable

5. Jalankan pada terminal/CMD flutter doctor

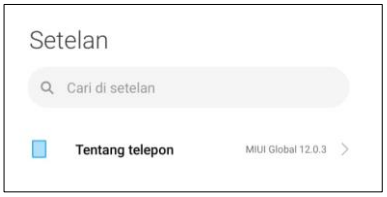


Gambar 2. 11 Flutter Doctor

2.7 Pengaturan Device Untuk Debugging

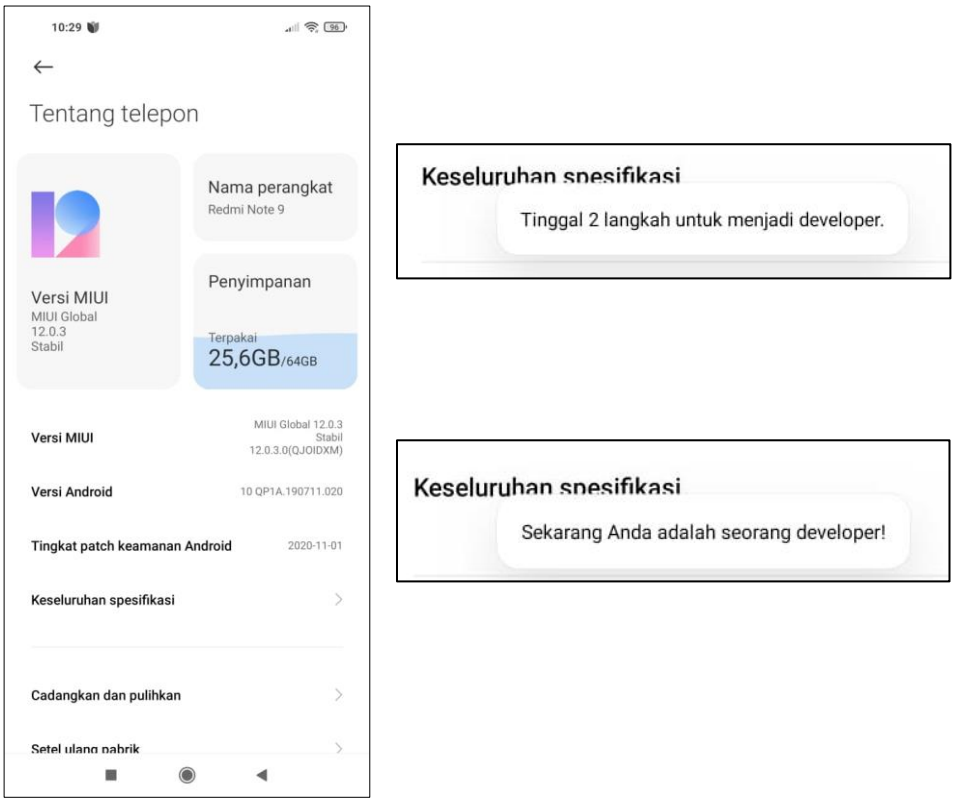
Sebelum melakukan proses debugging aplikasi Android secara langsung melalui perangkat fisik, pengguna perlu mengaktifkan mode pengembang (developer mode) dan USB debugging di perangkat Android. Berikut adalah langkah-langkah untuk mengatur device agar dapat digunakan untuk debugging:

a. Buka menu “Setelan” di perangkat Android.
b. Masuk ke Tentang Telepon.



Gambar 2. 12 Settings Android

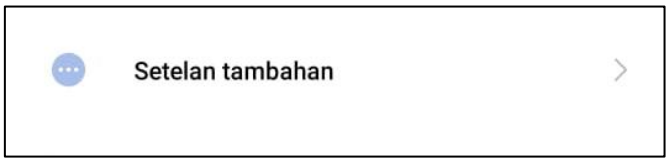
c. Temukan dan ketuk beberapa kali pada bagian Versi android(biasanya 7 kali) hingga muncul notifikasi bahwa Anda sudah menjadi developer:
"Tinggal 2 langkah untuk menjadi developer"
Kemudian: "Sekarang Anda adalah seorang developer"



Gambar 2. 13 Tentang Telepon

d. Kembali ke menu utama Setelan.

e. Pilih Setelan tambahan.



Gambar 2. 14 Setelan Tambahan

f. Di dalamnya, akan muncul menu baru yaitu Opsi pengembang, masuk ke menu tersebut.



Gambar 2. 15 Opsi Pengembang

g. Aktifkan Opsi pengembang dengan menggeser toggle ke posisi aktif.



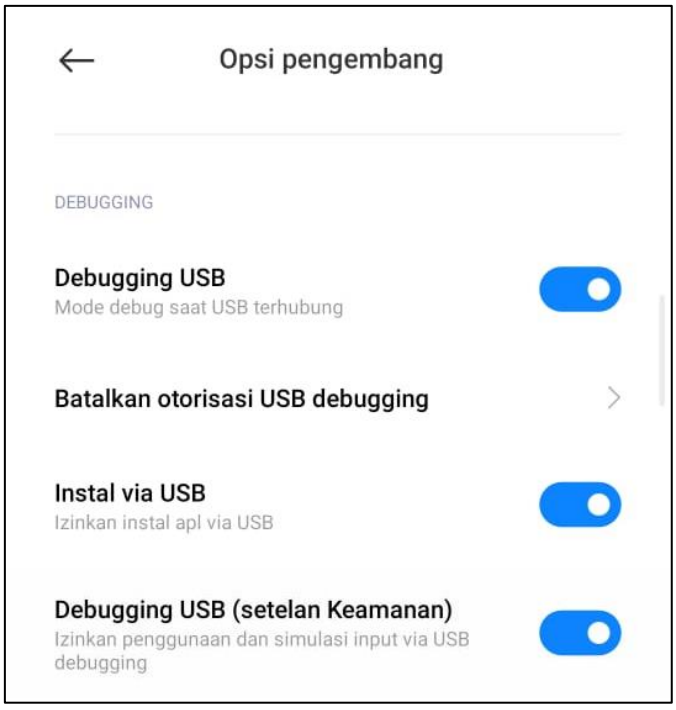
Gambar 2. 16 Aktifkan Opsi Pengembang

h. Aktifkan fitur Debugging USB dan aktifkan juga pengaturan tambahan seperti:

Batalkan otorisasi USB debugging

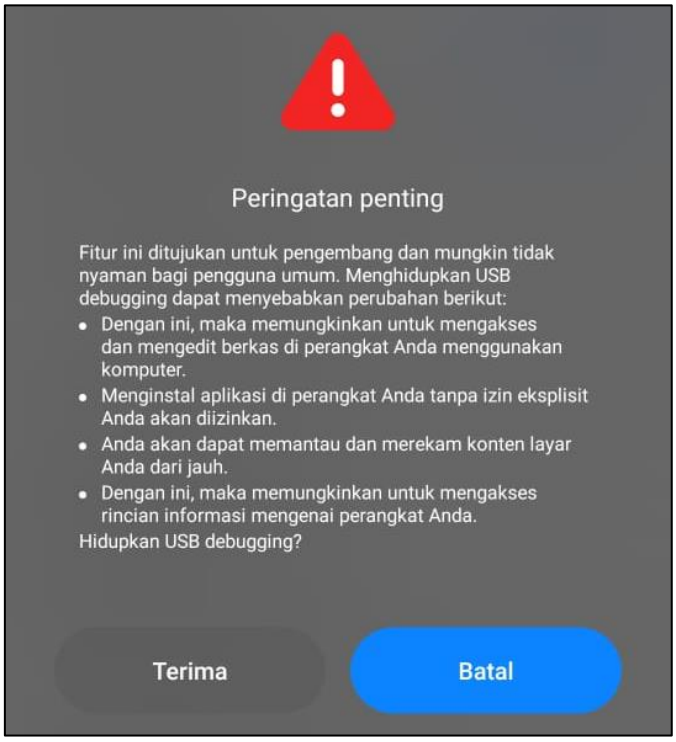
Instal via USB

Debugging USB (setelan keamanan)



Gambar 2. 17 Debugging USB

i. Saat muncul pop-up peringatan, tekan tombol Terima untuk mengonfirmasi.



Gambar 2. 18 Pop Up Peringatan

2.8 Membuat Project di Flutter

Setelah proses instalasi Flutter dan konfigurasi perangkat telah selesai dilakukan, tahap selanjutnya adalah mencoba membuat proyek Flutter pertama. Berikut adalah langkah-langkah dasar untuk membuat dan menjalankan project Flutter:

a. Buka terminal atau command prompt, lalu jalankan perintah berikut:

```
flutter create belajarflutter
```

Perintah ini akan membuat sebuah folder baru bernama belajarflutter yang berisi struktur dan file standar proyek Flutter.

b. Setelah proses pembuatan selesai, masuk ke direktori proyek tersebut dengan perintah:

```
cd belajarflutter
```

Perintah ini akan mengarahkan terminal ke folder belajarflutter, tempat semua file proyek berada.

c. Pastikan emulator atau perangkat Android sudah terhubung dan dalam mode debugging. Kemudian jalankan perintah berikut:

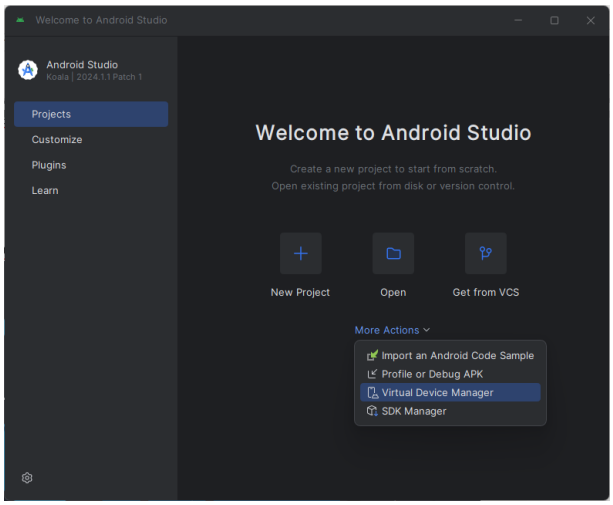
```
flutter run
```

CHAPTER 3
PENGENALAN FLUTTER DAN DART

3.1 Menyiapkan Emulator Android (Android Studio)

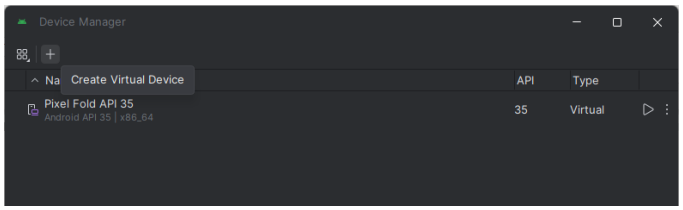
a. Buka Android Studio

b. Pilih Virtual Device Manager



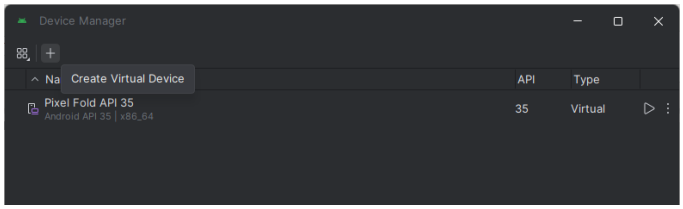
Gambar 3. 1 Virtual Device Manager

c. Klik Tombol + untuk membuat virtual device baru



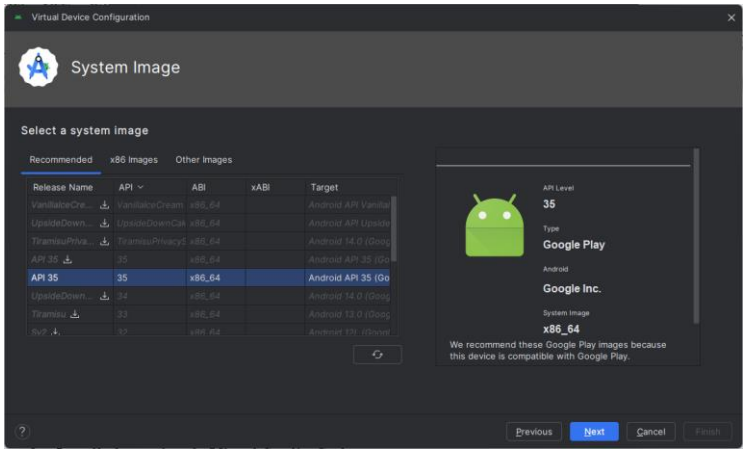
Gambar 3. 2 Tambah Device Baru

d. Pilih device sesuai keinginan, pada contoh kali ini pilih pixel 8, kemudian Next



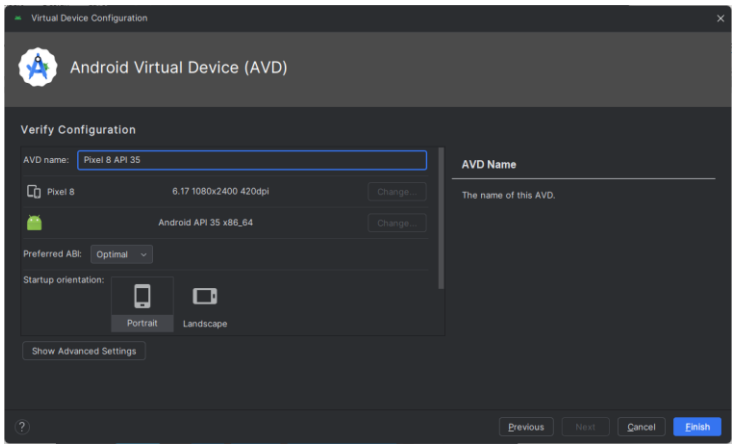
Gambar 3. 3 Pilih *Device*

e. Pilih API, pada contoh kali ini menggunkan API 35



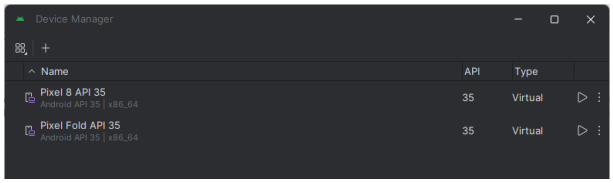
Gambar 3. 4 Pilih API

f. Beri nama AVD, kemudia pilih Finish



Gambar 3. 5 Nama AVD

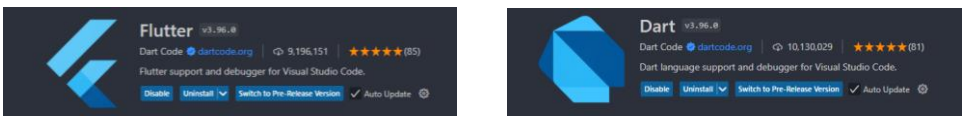
g. AVD baru telah ditambahkan



Gambar 3. 6 AVD Telah Ditambahkan

h. Jalankan AVD dengan klik icon start

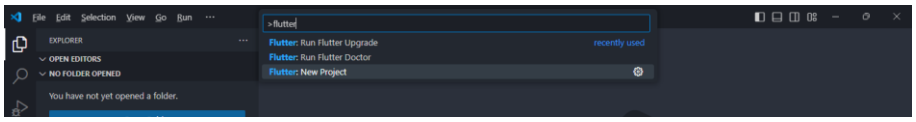
3.2 Install ekstensi pada Visual Studio Code



Gambar 3. 7 Install Ektensi

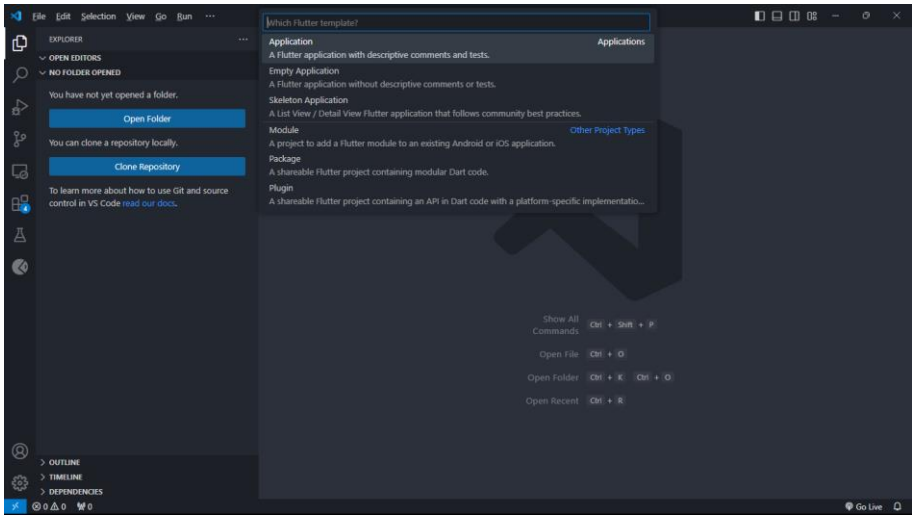
3.3 Membuat dan Menjalankan Project Flutter

a. Langkah selanjutnya, membuat dan menjalankan Flutter Project. Tekan F1, ketik “flutter” kemudian pilih New Project



Gambar 3. 8 New Project

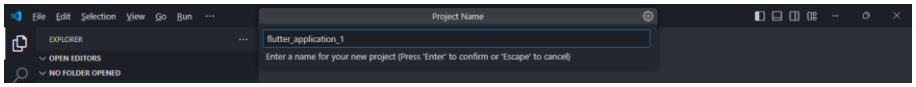
b. Pilih Application



Gambar 3. 9 Pilih Application

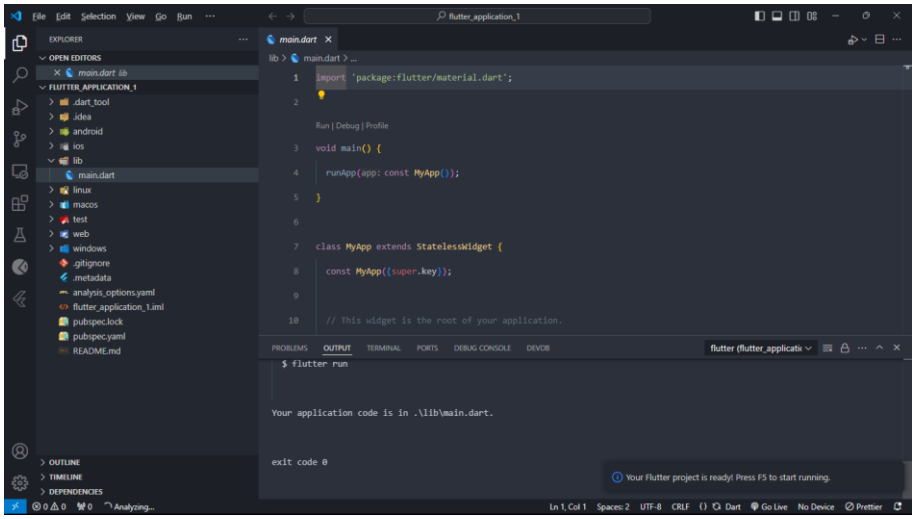
c. Pilih folder untuk menyimpan project flutter

d. Beri nama project



Gambar 3. 10 Beri Nama Project

e. Flutter berhasil di generate



Gambar 3. 11 Flutter

f. Tekan Ctrl+F5 atau run without Debug untuk menjalankan flutter

3.4 Pengenalan Bahasa Dart

3.3.1 Karakteristik Dart

a. Statically Typed

Dart adalah bahasa yang statically typed, kita perlu mendefinisikan variable sebelum digunakan.

```
var name = "Hello World";
print(name);
```

b. Type inference

Karakteristik ini dapat membuat suatu variable akan mengikuti tipe data dari valuenya. Namun, hal tersebut tidak berlaku jika kita sudah memberi tipe data pada variablenya.

```
// variable name secara otomatis jadi String
var name = "Hello World";
// tidak akan berubah ke int -> malah jadi error
String name = 10;
print(name);
```

c. String expressions

Kita dapat menggunakan tanda $ untuk menampilkan suatu variable bertipe apapun ke bentuk String. Hal ini dapat kita sebut sebagai String Interpolation.

```
var name = "Hello World";
String language = "Dart";
print("$name – bahasa yang digunakan : $language");
```

d. Object Oriented Programming

Dart berfokus pada paradigma OOP yang mana berorientasi pada object. Penggunaannya seperti apa ? kita akan bahas pada materi tentang OOP.

3.3.2 Tipe Data

a. String: Tipe data untuk menyimpan teks, misalnya "Flutter is Awesome!".
b. Bool: Tipe data untuk menyimpan nilai true atau false.
c. Num: Tipe umum untuk menyimpan angka, yang bisa berupa int (bilangan bulat) atau double (bilangan desimal).
d. Double: Tipe data untuk bilangan desimal seperti 5.9.
e. Int: Tipe data untuk bilangan bulat seperti 100.
f. List: Digunakan untuk menyimpan koleksi item yang berurutan. Dalam contoh ini, daftar buah.
g. Map: Koleksi key-value, digunakan untuk menyimpan pasangan nilai seperti nama siswa dan nilai mereka.
h. Dynamic: Tipe data yang fleksibel, di mana nilai dan tipe bisa berubah selama runtime.

```
void main() {
// 1. String: Tipe data untuk teks.
      String name = "Flutter is Awesome!";
      print("String: $name");

// 2. Bool: Tipe data untuk nilai boolean (true atau false).
      bool isFlutterAwesome = true;
      print("Bool: $isFlutterAwesome");

// 3. Num: Tipe data umum untuk angka, bisa berupa int atau
double.
      num age = 25; // Ini bisa berupa int atau double.
      print("Num: $age");

// 4. Double: Tipe data untuk angka desimal.
      double height = 5.9;
      print("Double: $height");

// 5. Int: Tipe data untuk bilangan bulat.
      int score = 100;
      print("Int: $score");

// 6. List: Koleksi terurut dari elemen, mirip dengan array.
      List<String> fruits = ["Apple", "Banana", "Orange"];
      print("List: $fruits");

// 7. Map: Koleksi pasangan key-value.
      Map<String, int> studentGrades = {
           "Alice": 90,
           "Bob": 85,
           "Charlie": 88
           };
      print("Map: $studentGrades");

// 8. Dynamic: Tipe data yang bisa berubah tipe saat
runtime.
      dynamic variable = "This is a string";
      print("Dynamic (String): $variable");
      variable = 123; // Bisa diubah ke tipe int.
      print("Dynamic (Int): $variable");

      variable = 3.14; // Bisa diubah ke tipe double.
      print("Dynamic (Double): $variable");
      variable = true; // Bisa diubah ke tipe bool.
      print("Dynamic (Bool): $variable");
      };
```

3.3.3 Final vs Const

Dalam Dart, terdapat dua keyword utama untuk membuat variabel yang bersifat *immutable* atau tidak dapat diubah: *final* dan *const*. Keduanya digunakan untuk mendeklarasikan variabel yang nilainya tidak bisa diubah setelah diinisialisasi, tetapi ada beberapa perbedaan penting antara keduanya.

a. *Final*

- Nilai ditentukan saat runtime (waktu eksekusi).
- Hanya dapat diinisialisasi sekali dan setelah itu tidak dapat diubah.

- Nilai dari variabel final dapat diberikan selama runtime, artinya bisa bergantung pada hasil fungsi atau nilai yang dihitung saat program berjalan.
- Digunakan untuk objek yang nilainya tetap tetapi bisa dinamis (bergantung pada waktu eksekusi).

```
void main() {
  final DateTime currentTime = DateTime.now(); // Ditentukan saat runtime
               print("Current time: $currentTime");
             }
```

Dalam contoh ini, currentTime diinisialisasi saat program dijalankan, karena DateTime.now() hanya dapat diketahui pada runtime.

b. *Const*

- Nilai harus ditentukan pada waktu kompilasi.
- Nilai dari variabel const harus sudah diketahui sebelum program dijalankan, yaitu pada waktu kompilasi.
- Nilai variabel const bersifat konstan dan immutable untuk seluruh durasi eksekusi program.
- Biasanya digunakan untuk nilai-nilai yang benar-benar tidak berubah, seperti angka atau string literal, atau untuk objek yang harus bersifat konstan.

```
void main() {
  const double pi = 3.14159; // Ditentukan saat kompilasi
  print("Value of pi: $pi");
}
```

Dalam contoh ini, nilai pi sudah diketahui pada waktu kompilasi, dan nilainya tidak akan pernah berubah selama program berjalan.

Perbedaan Utama:

a. *final*: Dapat diinisialisasi hanya sekali, tetapi inisialisasi bisa dilakukan saat runtime.
b. *const*: Nilai harus sudah diketahui pada waktu kompilasi, artinya semua ekspresi yang digunakan untuk menginisialisasi harus bersifat konstan.

3.3.4 Comment

Comment adalah bagian dari kode yang tidak akan dieksekusi. Komentar digunakan untuk memberikan penjelasan atau catatan dalam kode, agar mudah dipahami oleh pengembang lain maupun diri sendiri di masa depan.

Terdapat tiga jenis komentar di Dart:

a. Komentar satu baris:

Menggunakan // untuk memberikan komentar pendek.

b. Komentar multi-baris:

Menggunakan /* */ untuk komentar yang lebih panjang.

c. Komentar dokumentasi:

Menggunakan /// sebelum deklarasi fungsi atau class. Biasanya digunakan oleh dokumentasi otomatis.

```
// komentar 1
/*
   komentar x
   komentar Y
 */
/// komentar 2
/**
   *komentar x
   *komentar Y
 **/
```

3.3.5 Function

Function adalah blok kode yang dapat dipanggil untuk menjalankan tugas tertentu. Fungsi membantu membuat kode menjadi lebih modular dan mudah dikelola.

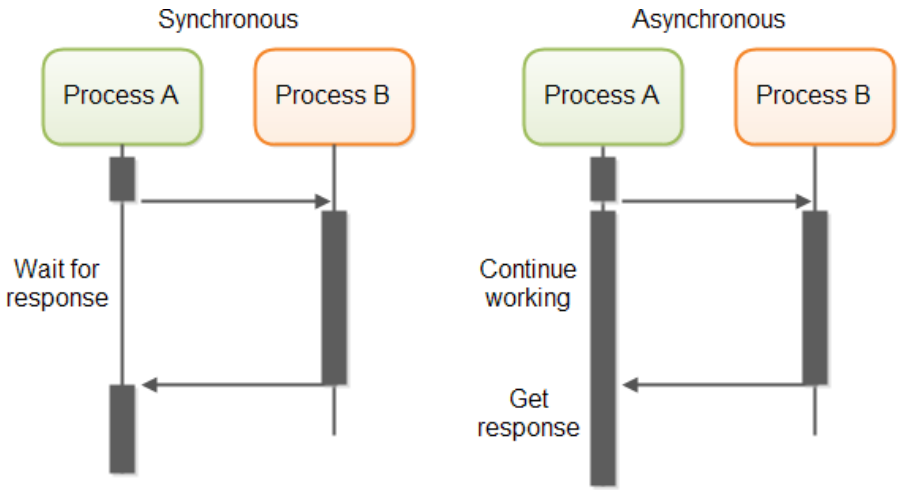
```
import 'dart:io';
// membuat fungsi luas persegi
int luasPersegi(int sisi) {
  return sisi * sisi;
}

main() {
  print("--- Program luas persegi ---");
  stdout.write("Input panjang sisi: ");
  int s = int.parse(stdin.readLineSync()!);

  // memanggil fungsi
       int hasil = luasPersegi(s);
```

3.3.6 Sync dan Async

Pada bahasa Dart, terdapat 2 jenis pengeksekusian kode. Kode akan dijalankan secara synchronous (sync) atau asynchronous (async). Berikut merupakan gambaran perbedaan keduanya :



Gambar 3. 12 Perbedaan Synchronous dan Asynchrounous

a. Sync (Synchronous):
   - *Operasi* dilakukan satu per satu, dengan instruksi berikutnya menunggu hingga instruksi sebelumnya selesai.
   - Cocok untuk operasi yang sangat cepat atau tidak melibatkan penundaan (seperti perhitungan sederhana atau manipulasi variabel).

b. Async (Asynchronous):
   - Operasi tidak menunggu operasi sebelumnya selesai, memungkinkan eksekusi kode lain berjalan tanpa hambatan.

- Cocok untuk operasi yang memakan waktu lama, seperti pengambilan data dari server, akses database, atau operasi I/O lainnya.
- Menggunakan Future dan await untuk menangani hasil operasi yang belum selesai.

```
void main() {
  print('Mulai');
  // Operasi sinkron: ini akan menunggu 3 detik
(blocking)
  String result = syncOperation();
  print(result);
  print('Selesai');
}
String syncOperation() {
  // Simulasi operasi yang memakan waktu lama
  return 'Operasi sinkron selesai';
}
```

```
void main() {
  print('Mulai');
  // Operasi asynchronous
  asyncOperation().then((result) {
    print(result);
  });
  print('Selesai');
}
Future<String> asyncOperation() async {
  // Simulasi operasi yang memakan waktu lama
  await Future.delayed(Duration(seconds: 3));
// Menunggu 3 detik secara asinkron
  return 'Operasi asinkron selesai';
}
```

3.3.7 Exception

Exception dalam Dart digunakan untuk menangani kondisi kesalahan (error) yang mungkin terjadi saat program dijalankan, seperti kesalahan pembagian dengan nol, file tidak ditemukan, dan sebagainya. Dart menyediakan blok try-catch untuk menangkap dan menangani kesalahan tersebut, agar program tidak langsung berhenti secara tiba-tiba.

a. Basic try-catch

Struktur try-catch dasar digunakan untuk menangkap kesalahan umum yang terjadi di dalam blok try. Ketika terjadi error, blok catch akan dijalankan dan menerima objek exception yang menjelaskan jenis kesalahan.

```
void main() {
  try {
    int result = 10 ~/ 0;
// Menghasilkan exception karena pembagian dengan nol.
    print(result);
  } catch (e) {
```

b. Try-catch dengan Stack Trace

Selain menangkap exception, catch juga dapat menerima informasi StackTrace, yaitu rincian jalur eksekusi kode hingga terjadinya kesalahan. Hal ini berguna untuk debugging, karena menunjukkan baris kode penyebab error.

```
void main() {
  try {
    int result = 10 ~/ 0; // Menghasilkan exception
    print(result);
  } catch (e, s) {
    print('Terjadi kesalahan: $e');
    print('Stack trace: $s');
// Menampilkan informasi detail tentang lokasi error
  }
```

c. Finally Block

Blok finally adalah blok opsional yang akan selalu dijalankan, baik ada exception ataupun tidak. Ini sering digunakan untuk membersihkan resource (misalnya menutup file atau koneksi).

```
 void main() {
   try {
     int result = 10 ~/ 0; // Menghasilkan exception
     print(result);
   } catch (e) {
     print('Terjadi kesalahan: $e');
   } finally {
     print('Ini akan selalu dijalankan');
   }
```

Blok finally akan selalu dijalankan setelah try dan catch selesai, tidak peduli apakah terjadi exception atau tidak.

d. Throw Keyword (Melempar Exception)

Keyword throw digunakan untuk memunculkan exception secara eksplisit berdasarkan kondisi tertentu. Teknik ini memungkinkan pengembang mendefinisikan validasi dan

logika error secara manual. Ini berguna ketika ingin memvalidasi kondisi tertentu dalam program dan melempar exception jika kondisi tersebut tidak terpenuhi.

```
void validateAge(int age) {
  if (age < 18) {
    throw Exception('Umur harus lebih dari 18 tahun');
  } else {
    print('Validasi berhasil');
  }
}
void main() {
  try {
    validateAge(16); // Ini akan menyebabkan exception
  } catch (e) {
    print('Error: $e');
  }
}
```

Fungsi validateAge memeriksa apakah umur kurang dari 18 tahun. Jika ya, ia melempar sebuah exception.

e. Membuat Exception Custom

Dart mendukung pembuatan exception khusus (custom exception) dengan mendefinisikan kelas baru yang mengimplementasikan interface Exception. Custom exception bermanfaat untuk memberikan penanganan error yang lebih spesifik dan sesuai konteks aplikasi.

```
class AgeException implements Exception {
  String errorMessage() {
    return 'Umur tidak valid';
  }
}
void validateAge(int age) {
  if (age < 18) {
    throw AgeException();
  } else {
    print('Validasi berhasil');
  }
}
void main() {
  try {
    validateAge(16); // Melempar AgeException
  } catch (e) {
    print('Error: ${e.errorMessage()}');
  }
```

Di sini, kita membuat kelas AgeException yang mengimplementasikan interface Exception. Saat umur tidak valid, kita melempar exception kustom ini.

3.3.8 Loop

Loop adalah cara untuk menjalankan blok kode berulang kali dalam sebuah program. Dart menyediakan beberapa jenis loop yang dapat digunakan untuk berbagai keperluan, seperti iterasi melalui elemen-elemen dari koleksi atau melakukan operasi yang berulang berdasarkan kondisi tertentu.

a. for loop

for loop digunakan ketika jumlah iterasi diketahui. Struktur ini terdiri dari tiga bagian utama: inisialisasi, kondisi, dan perubahan nilai. Cocok untuk kasus perulangan yang memerlukan kontrol penuh atas indeks.

```
for (initialization; condition; increment/decrement) {
  // Blok kode yang akan dijalankan berulang kali
}
```

```
void main() {
  for (int i = 0; i < 5; i++) {
    print('Iterasi ke-$i');
  }
```

Dalam contoh ini, loop for akan berjalan sebanyak 5 kali, mulai dari i = 0 hingga i < 5.

b. while loop

While loop menjalankan blok kode selama kondisi bernilai true. Kondisi diperiksa sebelum blok dijalankan. Umumnya digunakan saat jumlah iterasi tidak diketahui secara pasti.

```
while (condition) {
  // Blok kode yang akan dijalankan berulang kali
}
```

```
void main() {
  int i = 0;
  while (i < 5) {
    print('Iterasi ke-$i');
    i++;
// Menambah nilai i setiap kali loop dijalankan
  }
}
```

Loop ini akan terus berjalan selama kondisi i < 5 bernilai true.

c. do-while loop

Do-while loop hampir mirip dengan while loop, namun kondisi diperiksa setelah blok kode dijalankan. Dengan demikian, blok akan dijalankan setidaknya satu kali, meskipun kondisi bernilai false sejak awal.

```
do {
  // Blok kode yang akan dijalankan setidaknya sekali
} while (condition);
```

```
void main() {
  int i = 0;
      do {
    print('Iterasi ke-$i');
    i++;
  } while (i < 5);
}
```

Dalam contoh ini, blok kode di dalam do akan dijalankan terlebih dahulu, kemudian kondisi diperiksa. Loop ini akan berjalan sebanyak 5 kali.

d. for-in loop

For-in loop digunakan untuk iterasi langsung pada koleksi seperti List, Set, atau Map. Setiap elemen dalam koleksi akan diakses satu per satu tanpa perlu indeks.

```
for (var element in collection) {
  // Blok kode yang akan dijalankan untuk setiap elemen
}
```

```
void main() {
  List<String> fruits = ['Apple', 'Banana', 'Orange'];
  for (var fruit in fruits) {
    print('Buah: $fruit');
  }
}
```

Pada contoh ini, loop akan berjalan sebanyak jumlah elemen dalam List dan mencetak setiap elemen.

e. for each loop

ForEach adalah metode yang dimiliki oleh koleksi untuk melakukan iterasi. Fungsi atau lambda digunakan untuk mengeksekusi perintah pada setiap elemen koleksi secara deklaratif.

```
void main() {
  List<String> fruits = ['Apple', 'Banana', 'Orange'];
  fruits.forEach((fruit) {
    print('Buah: $fruit');
  });
`
```

Pada contoh ini, setiap elemen dalam List akan diproses oleh fungsi forEach.

f. Break and continue loop

Break digunakan untuk menghentikan eksekusi loop secara paksa sebelum kondisi selesai. Continue digunakan untuk melewatkan satu iterasi dan langsung lanjut ke iterasi berikutnya, tanpa menjalankan perintah yang tersisa dalam loop untuk iterasi tersebut.

```
void main() {
  for (int i = 0; i < 10; i++) {
    if (i == 5) {
      break; // Loop berhenti saat i sama dengan 5
    }
    print('Iterasi ke-$i');
  }
}
void main() {
  for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
      continue;
// Melewatkan iterasi saat i adalah bilangan genap
    }
    print('Iterasi ke-$i');
  }
}
```

Pada contoh ini, setiap kali nilai i adalah bilangan genap, perintah continue akan melewatkan iterasi tersebut dan melanjutkan ke iterasi berikutnya.

Kesimpulan:

- for: Loop dasar yang cocok ketika Anda mengetahui jumlah iterasi yang diperlukan.
- while: Digunakan ketika Anda tidak tahu jumlah iterasi sebelumnya, tetapi ingin melanjutkan loop selama kondisi tertentu benar.
- do-while: Mirip dengan while, tetapi selalu mengeksekusi blok kode setidaknya sekali.
- for-in: Digunakan untuk iterasi melalui elemen koleksi seperti List atau Map.
- forEach: Cara yang lebih deklaratif untuk melakukan iterasi melalui koleksi.
- break dan continue: Digunakan untuk mengontrol alur eksekusi di dalam loop.

3.3.9 Conditional Statement

a. If-else Statement

if / else digunakan untuk mengevaluasi kondisi logis secara langsung. Jika kondisi terpenuhi, blok kode dalam if dijalankan. Jika tidak, maka blok else dijalankan.

```
import 'dart:io';


main() {
  print("=== Login ===");
  stdout.write("Password: ");
  var password = stdin.readLineSync();
  String pass = password.toString().trim();
  // cek password yang diinputkan
  if (pass == "kopi") {
    print("Selamat datang bos!");
  } else {
    print("Kamu siapa? Pergi sana!");
  }
}
```

b. Switch Case Statement

switch digunakan ketika ada banyak kondisi berdasarkan satu nilai tertentu. Lebih efisien dan rapi dibandingkan if-else bertingkat untuk nilai yang setara (seperti hari, angka, karakter tertentu).

```
import 'dart:io';

main() {
  print("~~~ Quote Harian ~~~");
  stdout.write("Input hari: ");
  var hr = stdin.readLineSync();
  String hari = hr.toString().trim().toLowerCase();
  String quote;

  switch (hari) {
    case "senin":
      {
        quote = "Mari kita mulai dari senin";
        break;
      }
    case "selasa":
      {
        quote = "Selesaikan tugas, dan bersantailah";
        break;
      }
    case "rabu":
      {
        quote = "Serbu! hari ini penuh semangat!";
        break;
      }
    case "kamis":
      {
        quote = "Meski hujan gerimis, aku belajar coding";
        break;
      }
    case "jumat":
      {
        quote = "Jum'at berkah";
        break;
```

```
      }
    case "sabtu":
      {
        quote = "Tenangkan jiwa di hari akhir!";
        break;
      }
    case "minggu":
      {
        quote = "Selamat berlibur!";
        break;
      }
    default:
      {
        quote = "Hari yang anda masukan salah!";
      }
  }

  print(quote);
}
```

3.3.10 Collestions

Dalam Dart, collections adalah struktur data yang digunakan untuk menyimpan dan mengelola sekelompok elemen atau objek. Dart menyediakan beberapa jenis koleksi yang umum digunakan: List, Set, dan Map. Setiap jenis koleksi memiliki kegunaan dan karakteristik yang berbeda, dan mereka dapat digunakan sesuai dengan kebutuhan dalam program.

a. List

List adalah koleksi yang terurut dan dapat menyimpan elemen-elemen yang duplikat. Elemen di dalam List dapat diakses berdasarkan indeksnya (indeks dimulai dari 0).

```
void main() {
  // Membuat List
  List<int> numbers = [1, 2, 3, 4, 5];

  // Menambahkan elemen ke dalam List
  numbers.add(6);

  // Mengakses elemen dengan indeks
  print(numbers[0]); // Output: 1

  // Menghapus elemen dari List
  numbers.remove(2);

  // Mencetak semua elemen
  print(numbers); // Output: [1, 3, 4, 5, 6]
}
```

Fitur List:

- Memiliki urutan (indexable).
- Dapat menyimpan elemen duplikat.
- Dapat berubah ukurannya (growable list), kecuali menggunakan List.filled() yang ukurannya tetap.

b. Set

Set adalah koleksi yang tidak terurut dan hanya menyimpan elemen-elemen yang unik (tidak ada elemen duplikat). Set berguna jika Anda ingin memastikan bahwa tidak ada elemen yang muncul lebih dari satu kali.

```
void main() {
  // Membuat Set
  Set<String> fruits = {'Apple', 'Banana', 'Orange'};

  // Menambahkan elemen ke dalam Set
  fruits.add('Grape');
  fruits.add('Apple'); // Tidak akan ditambahkan karena
sudah ada

  // Menghapus elemen dari Set
  fruits.remove('Banana');

  // Mencetak semua elemen
  print(fruits); // Output: {Apple, Orange, Grape}
}
```

Fitur Set:

- Tidak memiliki urutan tertentu.
- Hanya menyimpan elemen yang unik (tidak ada duplikat).
- Dapat menambah dan menghapus elemen secara dinamis.

c. Map

Map adalah koleksi yang menyimpan pasangan key-value. Setiap key dalam Map harus unik, tetapi nilai (value) dapat memiliki duplikat. Map sering digunakan untuk menyimpan data yang memiliki hubungan key-value, seperti kamus atau data JSON.

```
void main() {
  // Membuat Map
  Map<String, int> studentScores = {
    'Alice': 85,
    'Bob': 90,
    'Charlie': 88
  };

  // Menambahkan key-value baru
  studentScores['Dave'] = 92;

  // Mengakses nilai berdasarkan key
  print(studentScores['Alice']); // Output: 85

  // Menghapus key-value dari Map
  studentScores.remove('Charlie');

  // Mencetak semua key-value
  print(studentScores);
// Output: {Alice: 85, Bob: 90, Dave: 92}
}
```

Fitur Map:

- Setiap elemen disimpan sebagai pasangan key-value.
- Key harus unik, tetapi nilai tidak harus unik.
- Dapat digunakan untuk menyimpan data yang memiliki hubungan identifikasi antara key dan value.

3.3.11 Null Safety

Null safety di Dart adalah fitur yang dirancang untuk mencegah kesalahan null reference, yang sering terjadi ketika kode mencoba mengakses atau menggunakan nilai yang null. Dengan null safety, Dart memperkenalkan konsep tipe yang lebih aman, memungkinkan pengembang untuk menentukan apakah variabel dapat bernilai null atau tidak.

a. Konsep Dasar Null Safety

- Tipe Non-nullable: Secara default, semua tipe di Dart adalah non-nullable. Artinya, variabel yang dideklarasikan tidak dapat memiliki nilai null.

```
int number = 10; // Non-nullable
number = null; // Error: A value of type 'Null' can't be
assigned to a variable of type 'int'
```

- Tipe Nullable: Jika Anda ingin variabel dapat memiliki nilai null, Anda harus menambahkan tanda tanya (?) setelah tipe data.

```
int? nullableNumber; // Nullable
nullableNumber = null; // Valid
```

- Late Initialization: Jika Anda ingin mendeklarasikan variabel non-nullable tetapi tidak ingin menginisialisasinya segera, Anda bisa menggunakan kata kunci late. Ini memberi tahu Dart bahwa variabel akan diinisialisasi sebelum digunakan.

```
late String name;
name = 'John Doe'; // Inisialisasi kemudian
```

b. Menggunakan Null Safety

- Memeriksa Null dengan Operator Null-aware
  1) ? (Operator akses null-aware): Mengizinkan akses ke anggota dari objek yang mungkin null.
  2) ?? (Operator null-coalescing): Mengembalikan nilai di sebelah kiri jika tidak null, jika tidak, mengembalikan nilai di sebelah kanan.
  3) ! (Operator non-null assertion): Memaksa Dart untuk menganggap nilai bukan null.

```
void main() {
  String? name;
  // Menggunakan operator null-aware
  print(name?.length); // Output: null
  // Menggunakan null-coalescing
  print(name ?? 'Guest'); // Output: Guest
  // Menggunakan non-null assertion
  name = 'Alice';
  print(name!.length); // Output: 5
}
```

- Fungsi dan Parameter

  Anda dapat mendeklarasikan parameter fungsi sebagai nullable atau non-nullable.

```
void greet(String? name) {
  print('Hello, ${name ?? 'Guest'}!');
}
void main() {
  greet(null); // Output: Hello, Guest!
  greet('Alice'); // Output: Hello, Alice!
}
```

- Penggunaan di Koleksi

  Anda juga bisa memiliki koleksi dengan elemen nullable.

```
void main() {
  List<int?> numbers = [1, 2, null, 4];

  for (var number in numbers) {
    print(number ?? 0); // Output: 1, 2, 0, 4
  }
}
```

Null safety meningkatkan keamanan kode dengan mencegah kesalahan yang disebabkan oleh null.

Tipe default adalah non-nullable, dan Anda perlu menggunakan ? untuk nullable.

Operator null-aware dan null-coalescing memudahkan pengelolaan nilai yang mungkin null.

Memanfaatkan null safety dapat membantu Anda menulis kode yang lebih aman dan lebih mudah dipelihara.

3.5 Object Oriented Programming (OOP) Dart

3.5.1 Konsep OOP

OOP merupakan sebuah paradigma pemrogramman berorientasi object. Hal ini bertujuan agar project kita lebih mudah untuk digunakan dan juga untuk dipelihara. Jika dianalogikan konsep OOP seperti apa, simak gambar berikut ini:



Gambar 3. 13 Ilustrasi OOP

Jika dilihat dari gambar di atas, gambar tersebut merupakan analogi suatu project yang kita buat tanpa berdasarkan orientasi pada suatu object. Kita tidak memisahkan baju, celana, kaos kaki, kerudung / hijab dll. Baju merupakan sebuah object, celana adalah object, kaos kaki juga object, dan seterusnya. Jika kita menatanya berdasarkan kategori jenisnya (object), maka akan terlihat lebih indah, rapi, dan mudah untuk dicari seperti gambar berikut ini :



Gambar 3. 14 Ilustrasi OOP 2

3.5.2 Class & Object

Di Dart, class adalah blueprint atau template untuk membuat objek. Class digunakan untuk mendefinisikan struktur dan perilaku dari objek. Flutter, yang menggunakan Dart sebagai bahasa pemrogramannya, sangat bergantung pada penggunaan class, terutama dalam mendefinisikan UI dan logika aplikasi.

a. Membuat Class

Sebuah class di Dart dapat berisi fields (property/atribut) dan methods (fungsi yang berhubungan dengan class). Anda juga dapat membuat constructor untuk menginisialisasi objek.

```
class Person {
  // Fields (atribut atau property)
  String name;
  int age;
  // Constructor untuk menginisialisasi objek
  Person(this.name, this.age);
  // Method
  void introduce() {
    print('Hi, my name is $name and I am $age years old.');
  }
}
void main() {
  // Membuat objek dari class Person
  Person person = Person('Alice', 25);
  person.introduce(); // Output: Hi, my name is Alice and
I am 25 years old.
}
```

b. Fields dan Methods

Fields adalah variabel yang didefinisikan di dalam class dan digunakan untuk menyimpan informasi terkait objek tersebut. Methods adalah fungsi yang didefinisikan di dalam class yang menentukan perilaku objek.

```
class Car {
  String brand;
  int year;
  // Constructor
  Car(this.brand, this.year);
  // Method
  void displayInfo() {
    print('Car: $brand, Year: $year');
  }
}
void main() {
  Car car = Car('Toyota', 2020);
  car.displayInfo(); // Output: Car: Toyota, Year: 2020
}
```

3.5.3 Constructor

Constructor adalah metode khusus yang digunakan untuk menginisialisasi objek. Dalam Dart, ada beberapa jenis constructor, termasuk default constructor, named constructor, dan factory constructor.

```
class Animal {
  String name;
  // Default constructor
  Animal(this.name);
  void makeSound() {
    print('$name makes a sound');
  }
}
void main() {
  Animal cat = Animal('Cat');
  cat.makeSound(); // Output: Cat makes a sound
}
```

Named Constructor: Digunakan untuk memberikan nama pada constructor untuk keperluan inisialisasi yang lebih spesifik.

```
class Rectangle {
  double width;
  double height;

  // Named constructor
  Rectangle.square(double size) : width = size, height
= size;

  double area() => width * height;
}

void main() {
  Rectangle square = Rectangle.square(4);
  print('Square area: ${square.area()}'); // Output:
Square area: 16
}
```

3.5.4 Inheritance

Inheritance adalah konsep OOP yang memungkinkan satu class untuk mewarisi properti dan metode dari class lain. Dart mendukung pewarisan dengan menggunakan keyword extends.

```
class Animal {
  String name;

  Animal(this.name);

  void makeSound() {
    print('$name makes a sound');
  }
}

// Class Dog mewarisi class Animal
class Dog extends Animal {
  Dog(String name) : super(name);

  // Overriding method makeSound
  @override
  void makeSound() {
    print('$name barks');
  }
}

void main() {
  Dog dog = Dog('Rex');
  dog.makeSound(); // Output: Rex barks
}
```

3.5.5 Getter & Setter

Getters dan setters memungkinkan Anda mengontrol akses terhadap field dari luar class, biasanya digunakan untuk validasi atau logika tambahan.

```
 class Circle {
   double _radius;

   Circle(this._radius);

   // Getter untuk mendapatkan nilai area
   double get area => 3.14 * _radius * _radius;

   // Setter untuk mengatur nilai radius dengan validasi
   set radius(double value) {
     if (value <= 0) {
       print('Radius harus lebih besar dari 0');
     } else {
       _radius = value;
     }
   }
 }
```

```
void main() {
  Circle circle = Circle(5);
  print('Area: ${circle.area}'); // Output: Area: 78.5

  circle.radius = 10;
  print('Updated Area: ${circle.area}'); // Output: Updated Area:
314.0
}
```

3.5.6 Abstract Class

Abstract class adalah class yang tidak dapat diinstansiasi dan biasanya digunakan sebagai base class. Abstract class dapat memiliki method abstract yang harus diimplementasikan oleh subclass.

```
abstract class Shape {
  // Method abstract
  double getArea();

  void display() {
    print('Shape has an area of ${getArea()}');
  }
}

// Subclass harus mengimplementasikan method
getArea
class Rectangle extends Shape {
  double width, height;

  Rectangle(this.width, this.height);

  @override
  double getArea() => width * height;
}

void main() {
  Rectangle rectangle = Rectangle(5, 10);
  rectangle.display(); // Output: Shape has an
area of 50.0
}
```

3.6 Flutter Dasar

Struktur folder dan file utama dalam proyek Flutter:

a. Folder lib/

Ini adalah folder utama tempat Kita akan menulis kode aplikasi Flutter. File utama yang biasanya ada di sini adalah:

- main.dart: File ini adalah entry point dari aplikasi Flutter. Di sini, biasanya terdapat fungsi *main()* yang memanggil *runApp()* untuk menjalankan aplikasi.
- Kita dapat membuat sub-folder lain di dalam *lib/* untuk mengorganisir kode seperti widgets, screens, models, services, dll.

b. Folder android/

Folder ini berisi kode dan konfigurasi untuk platform Android. Jika Kita ingin menambahkan fitur khusus Android (misalnya, native code dalam Kotlin atau Java), Kita akan bekerja di sini.

- build.gradle: Konfigurasi build untuk aplikasi Android Kita.
- AndroidManifest.xml: File konfigurasi yang mendefinisikan informasi aplikasi Android, seperti izin yang diperlukan oleh aplikasi Kita.

c. Folder ios/

Folder ini berisi kode dan konfigurasi untuk platform iOS. Untuk mengintegrasikan kode native ke dalam iOS (dalam Swift atau Objective-C), Kita akan mengedit file di sini.

- Info.plist: File konfigurasi aplikasi iOS yang menentukan izin dan pengaturan aplikasi di sistem iOS.
- Podfile: File untuk mengelola dependensi CocoaPods.

d. Folder test/

Folder ini berisi tes unit dan widget untuk aplikasi Flutter Kita. Tes ini membantu memastikan bahwa aplikasi Kita berjalan dengan benar.

- widget_test.dart: Biasanya ini adalah file tes bawaan yang dibuat secara otomatis saat Kita membuat proyek Flutter.

e. Folder build/

Folder ini dihasilkan secara otomatis oleh Flutter selama proses build. Kita tidak perlu melakukan perubahan pada folder ini karena isinya dihasilkan oleh sistem build Flutter berdasarkan kode dan pengaturan yang Kita miliki.

f. Folder web/

Jika Kita menargetkan aplikasi untuk web, folder ini akan berisi file yang terkait dengan proyek Flutter Web. Folder ini memiliki konfigurasi untuk menjalankan aplikasi di browser.

g. Folder macos/, windows/, linux/

Jika Kita menargetkan aplikasi desktop, folder ini berisi kode dan konfigurasi untuk platform macOS, Windows, atau Linux. Kita dapat menggunakan folder ini untuk menambahkan kode native yang spesifik untuk masing-masing platform.

h. File pubspec.yaml

Ini adalah salah satu file paling penting di proyek Flutter Kita. File ini digunakan untuk mendefinisikan dependensi (library dan package), asset (gambar, font), dan informasi metadata tentang aplikasi Kita (seperti nama aplikasi, versi, dan deskripsi).

- Dependencies: Bagian ini digunakan untuk menambahkan paket atau library dari pub.dev yang akan digunakan dalam proyek Kita.
- Assets: Menentukan asset seperti gambar atau font yang digunakan dalam aplikasi.

i. File .gitignore

File ini digunakan untuk mengabaikan file dan folder tertentu dari sistem kontrol versi Git, seperti folder build/ atau file sementara lainnya yang tidak perlu dimasukkan ke dalam repository Git.

j. File README.md

File ini biasanya digunakan untuk memberikan penjelasan umum tentang proyek Kita. Ini adalah file teks markdown yang biasanya memberikan informasi tentang bagaimana menjalankan proyek, dependensi yang diperlukan, atau instruksi lainnya.

k. File analysis_options.yaml

File ini digunakan untuk mengkonfigurasi linter dan aturan analisis statis untuk kode Dart Kita. Kita dapat menyesuaikan aturan analisis seperti gaya kode, penggunaan variabel, dan hal-hal lain yang terkait dengan kualitas kode.

CHAPTER 4
MANAJEMEN LAYOUT DAN WIDGET PADA FLUTTER

4.1 Pengenalan Widget dalam Flutter

Semua Class pada Flutter bisa kita sebut sebagai widget. Kalau dianalogikan, jika di meja kita sekarang ada secangkir kopi, maka kopi tersebut adalah widget. Namun, perlu diperhatikan dari secangkir kopi tersebut, bahwa itu semua terdiri dari widget Cangkir, widget Kopi, widget Air, dan widget Gula. Oleh karena itu, untuk menjadi Flutter Developer Sejati, kita harus mengetahui beragam widget sebanyak-banyaknya.

Salah satu widget paling penting adalah MaterialApp, yang berfungsi sebagai wadah utama dalam membangun tampilan aplikasi. Jika dianalogikan, MaterialApp seperti papan kayu yang menjadi dasar untuk melukis seluruh antarmuka aplikasi.



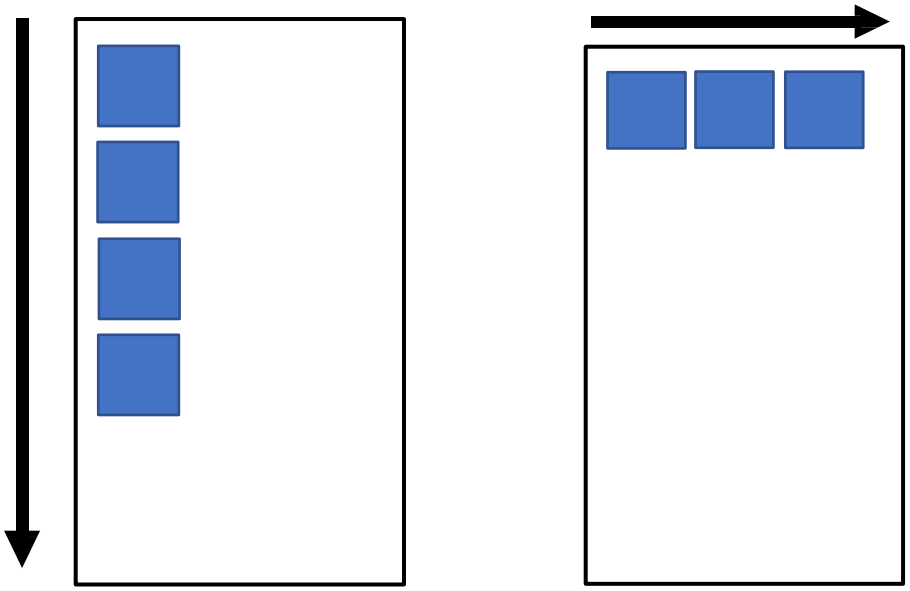
Gambar 4. 1 Papan Kayu

Setelah itu, widget Scaffold digunakan sebagai kerangka utama dalam setiap halaman aplikasi. Scaffold dapat diibaratkan sebagai kanvas tempat kita melukis di atas papan kayu tersebut.



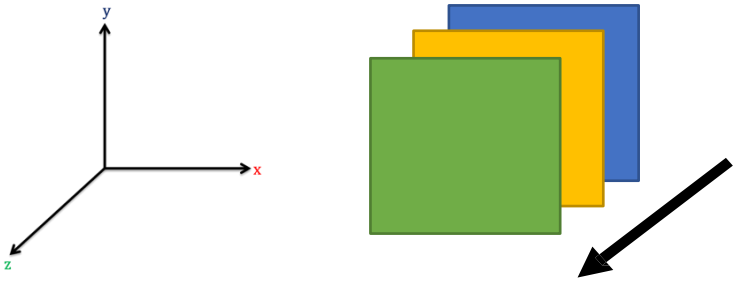
Gambar 4. 2 Kanvas

Untuk menampilkan konten dalam berbagai arah, Flutter menyediakan beberapa widget layouting. Misalnya, Column digunakan untuk menyusun widget secara vertikal dari atas ke bawah, sedangkan Row digunakan untuk menyusun widget secara horizontal dari kiri ke kanan.



Gambar 4. 3 Row dan Column

Selain itu, terdapat juga widget Stack, yang memungkinkan kita menempatkan widget secara bertumpuk pada sumbu Z (depan-belakang), sehingga dapat menciptakan efek tumpang tindih antar elemen tampilan.



Gambar 4. 4 Sumbu dan Layer Widget

4.2 Widget Utama pada Flutter

4.2.1 Material App

MaterialApp adalah titik awal untuk aplikasi berbasis Material Design. Widget ini mendefinisikan pengaturan umum aplikasi seperti tema, navigasi, dan title aplikasi.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';


void main(){
```

```
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text("Hallo Dunia"),
        ),
      ),
    );
}
}
```

4.2.2 Scaffold

Scaffold adalah struktur dasar yang menyediakan kerangka layout untuk aplikasi. Scaffold memberi beberapa fitur seperti AppBar, drawer, body, bottomNavigationBar, dan lainnya.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';

void main(){
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text("Hallo Dunia"),
        ),
      ),
    );
}
}
```

Contoh Lain

```
Scaffold(
  appBar: AppBar(
    title: Text('Scaffold Example'),
  ),
  body: Center(child: Text('Content Area')),
  floatingActionButton: FloatingActionButton(
    onPressed: () {},
    child: Icon(Icons.add),
  ),
);
```

4.2.3 AppBar

AppBar digunakan sebagai bar navigasi di atas layar. Biasanya menampilkan title aplikasi dan berisi beberapa aksi seperti tombol atau menu.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';

void main(){
  runApp(KelasBebas());
}
class KelasBebas extends StatelessWidget {
  const KelasBebas({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          backgroundColor: Colors.red[900],
          title: Text("Aplikasi Pertamaku"),
          centerTitle: true,
        ),
        body: Center(
          child: Text("Hello world"),
        ),
      ),
    );
  }
}
```

4.2.4 Container

Container adalah widget serbaguna yang bisa digunakan untuk styling, memberikan padding, margin, dan layout dasar lainnya.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';

void main(){
  runApp(KelasBebas());
}

class KelasBebas extends StatelessWidget {
  const KelasBebas({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          backgroundColor: Colors.red[900],
          title: Text("Aplikasi Pertamaku"),
          centerTitle: true,
        ),
        body: Center(
          child: Container(
            width: 200,
            height: 200,
            color: Colors.blue,
            child: Center(
              child: Text("Hello world"),
            ),
          ),
        ),
      ),
    );
  }
}
```

Contoh Lain

```
Container(
```

```
  padding: EdgeInsets.all(16.0),
  margin: EdgeInsets.all(8.0),
  decoration: BoxDecoration(
    color: Colors.blue,
    borderRadius: BorderRadius.circular(8.0),
  ),
  child: Text('This is a Container'),
);
```

4.3 Widget Dasar (Visible Widget)

Visible Widget merupakan widget yang memiliki tampilan fisik di layar dan bisa dilihat atau berinteraksi langsung oleh pengguna. Contohnya seperti teks, gambar, ikon, dan tombol. Widget ini berfungsi sebagai komponen utama dari antarmuka pengguna yang langsung ditampilkan.

4.3.1 Text

Text digunakan untuk menampilkan teks di layar aplikasi. Widget ini mendukung berbagai gaya teks seperti ukuran huruf, warna, font, dan alignment.

Contoh Penggunaan:

```
Text(
  'Hello, Flutter!',
  style: TextStyle(fontSize: 24, color: Colors.black),
);
```

4.3.2 Flutter Logo

FlutterLogo adalah widget khusus yang menampilkan logo resmi Flutter. Biasanya digunakan untuk tujuan branding atau edukasi, terutama dalam tampilan awal aplikasi atau halaman demo.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';

void main() {
  runApp(KelasBebas());
}

class KelasBebas extends StatelessWidget {
  const KelasBebas({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
      home: Scaffold(
        appBar: AppBar(
          backgroundColor: Colors.red[900],
          title: Text("Aplikasi Pertamaku"),
          centerTitle: true,
        ),
        body: Center(
            child: FlutterLogo(
          size: 200,
        )),
      ),
    );
  }
}
```

Contoh Lain

```
FlutterLogo(
  size: 100,
  style: FlutterLogoStyle.markOnly,
);
```

4.3.3 ElevatedButton

ElevatedButton adalah tombol berbasis Material Design yang memiliki efek bayangan (elevasi). Tombol ini digunakan untuk menangani aksi saat ditekan oleh pengguna.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';

void main() {
  runApp(KelasBebas());
}

class KelasBebas extends StatelessWidget {
  const KelasBebas({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          backgroundColor: Colors.red[900],
          title: Text("Aplikasi Pertamaku"),
          centerTitle: true,
```

```
        ),
        body: Center(
            child: ElevatedButton(
          onPressed: () {
            print('Button Pressed');
          },
          child: Text('Press Me'),
        )),
      ),
    );
  }
}
```

4.3.4 Icon

Icon digunakan untuk menampilkan ikon grafis, seperti simbol panah, rumah, atau ikon sistem lainnya. Umumnya digunakan untuk memperjelas fungsi dari tombol atau fitur tertentu.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';

void main() {
  runApp(KelasBebas());
}

class KelasBebas extends StatelessWidget {
  const KelasBebas({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          backgroundColor: Colors.red[900],
          title: Text("Aplikasi Pertamaku"),
          centerTitle: true,
        ),
        body: Center(
            child: Icon(
              Icons.home,
              size: 200,
              color: Colors.red[900],)
```

```
            ),
      ),
    );
  }
}
```

4.3.5 Image

Image digunakan untuk menampilkan gambar dari berbagai sumber, baik dari file lokal (asset), dari internet (network), maupun dari memori aplikasi. Widget ini sangat penting dalam membangun antarmuka visual yang menarik.

Contoh Penggunaan:

```
import 'package:flutter/material.dart';

void main() {
  runApp(KelasBebas());
}

class KelasBebas extends StatelessWidget {
  const KelasBebas({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          backgroundColor: Colors.red[900],
          title: Text("Aplikasi Pertamaku"),
          centerTitle: true,
        ),
        body: Center(
          // Image Provider
          // 1. Asset Image
          // buat folder (asset) pada root project
          // pindahkan gambar ke folder (asset)
          // daftarkan gambar pada pubspec.yaml
          child: Image.asset('images/robot.jpg'),
        ),
      ),
    );
  }
}
```

Contoh Lain

```
//2. Network image
//diambil dari internet
Image.network(
  'https://flutter.dev/images/flutter-logo-sharing.png',
  width: 100,
  height: 100,
);
```

Penjelasan Ringkas Setiap Widget

- MaterialApp: Merupakan dasar dari aplikasi Flutter berbasis Material Design.
- Scaffold: Menyediakan struktur dasar untuk aplikasi, seperti AppBar, body, drawer.
- AppBar: Menyediakan bilah navigasi di atas layar.
- Container: Widget untuk mengatur tata letak, margin, padding, dan dekorasi.
- Text: Menampilkan teks.
- FlutterLogo: Menampilkan logo Flutter.
- ElevatedButton: Tombol untuk interaksi pengguna.
- Icon: Menampilkan ikon.
- Image: Menampilkan gambar dari URL, assets, atau lainnya.

4.4 Widget Layout (Invisible Widgets/Layouting)

Invisible Widget atau **Layouting Widget** adalah widget yang tidak memiliki tampilan secara langsung di layar, namun sangat penting untuk mengatur struktur dan penempatan elemen-elemen lain dalam aplikasi. Widget ini digunakan untuk menyusun posisi, arah, dan tata letak dari berbagai elemen UI agar tampil rapi dan terorganisir.

4.4.1 Center

Digunakan untuk menempatkan widget anak tepat di tengah area yang tersedia, baik secara horizontal maupun vertikal.

Contoh Penggunaan:

```
Center(
  child: Text('Centered Text'),
);
```

4.4.2 Column

Digunakan untuk menyusun beberapa widget secara vertikal dari atas ke bawah. Cocok ketika ingin menampilkan konten dalam urutan kolom.

Contoh Penggunaan:

```
Column(
  children: [
    Text('Item 1'),
    Text('Item 2'),
    Text('Item 3'),
  ],
);
```

4.4.3 Row

Berfungsi menyusun beberapa widget secara horizontal dari kiri ke kanan. Umumnya digunakan untuk tata letak berdampingan secara sejajar.

Contoh Penggunaan:

```
Row(
  children: [
    Icon(Icons.star),
    Text('Row Text'),
  ],
);
```

4.4.4 Stack

Memungkinkan beberapa widget ditumpuk di atas satu sama lain seperti lapisan. Widget akan ditampilkan berdasarkan urutan penulisan, dari belakang ke depan.

Contoh Penggunaan:

```
Stack(
  children: [
    Container(
      width: 100,
      height: 100,
      color: Colors.red,
    ),
    Positioned(
      top: 20,
      left: 20,
      child: Icon(Icons.star, color: Colors.white),
    ),
  ],
);
```

4.4.5 Single Child Scroll View

Membuat konten dapat digulir (scroll) jika ukurannya melebihi layar. Hanya bisa menampung satu widget anak, biasanya digunakan bersama Column untuk tampilan vertikal.

```
SingleChildScrollView(
  child: Column(
    children: List.generate(20, (index) => Text('Item
$index')),
  ),
);
```

4.4.6 List View

ListView adalah widget scrollable yang sangat umum digunakan untuk menampilkan daftar item. Biasanya digunakan untuk menampilkan data dalam bentuk daftar yang panjang.

Contoh Penggunaan:

```
ListView(
  children: [
    ListTile(title: Text('Item 1')),
    ListTile(title: Text('Item 2')),
    ListTile(title: Text('Item 3')),
  ],
);
```

4.4.7 Grid View

Digunakan untuk menyusun item dalam format grid atau kotak-kotak yang teratur, baik dalam dua kolom atau lebih, tergantung pada konfigurasi yang diatur.

```
GridView.count(
  crossAxisCount: 2,
  children: [
    Container(color: Colors.red, child: Text('Item 1')),
    Container(color: Colors.blue, child: Text('Item 2')),
    Container(color: Colors.green, child: Text('Item 3')),
    Container(color: Colors.yellow, child: Text('Item 4')),
  ],
);
```

4.5 Ringkasan dan Kesimpulan

4.5.1 Perbedaan Visible vs Invisible Widget

Visible widget adalah widget yang tampak langsung oleh pengguna di antarmuka aplikasi, seperti teks, gambar, ikon, dan tombol.

Invisible widget (layouting widget) tidak terlihat secara langsung, namun berperan penting dalam mengatur posisi dan struktur tampilan dari visible widget. Contohnya seperti Center, Column, Row, dan Stack.

4.5.2 Pentingnya Layouting dalam Flutter

Layouting sangat penting karena menentukan bagaimana elemen-elemen UI disusun dan ditampilkan di layar. Tanpa layout yang tepat, antarmuka akan terlihat berantakan, sulit digunakan, dan tidak responsif di berbagai ukuran layar. Layouting widget membantu developer menciptakan struktur tampilan yang rapi, teratur, dan konsisten.

4.5.3 Tips Mengelola Layout secara Efisien

- Gunakan widget layout sesuai kebutuhan (misalnya Column untuk vertikal, Row untuk horizontal).
- Hindari penggunaan widget layout bersarang terlalu dalam (nested layout) karena bisa menurunkan performa.
- Gunakan Expanded, Flexible, atau Spacer untuk mengatur ruang antar widget dengan lebih dinamis.
- Manfaatkan widget seperti SingleChildScrollView untuk menghindari overflow pada layar kecil.
- Preview tampilan pada berbagai ukuran layar menggunakan emulator atau perangkat berbeda untuk memastikan responsivitas.

CHAPTER 5
MANAJEMEN DAN NAVIGASI STATE

5.1 Stateless vs Stateful Widget

5.1.1 Stateless Widget

StatelessWidget adalah jenis widget yang bersifat statis, artinya tidak memiliki state atau data internal yang dapat berubah saat aplikasi dijalankan. Cocok digunakan untuk tampilan yang tidak membutuhkan interaksi atau perubahan dinamis, seperti teks statis, ikon, atau layout tetap. Widget ini hanya dibangun sekali dan tidak akan diperbarui kecuali seluruh widget di-*rebuild*. Contoh penggunaannya adalah tampilan halaman informasi sederhana atau label teks tetap.

Contoh StatelessWidget:

```
class MyStatelessWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Stateless Widget'),
      ),
      body: Center(
        child: Text('This is a stateless widget'),
      ),
    );
  }
}
```

5.1.2 Stateful Widget

StatefulWidget memiliki state yang bisa berubah sesuai interaksi pengguna. Artinya, widget ini dapat memperbarui tampilannya secara dinamis ketika terjadi perubahan pada data internal. Misalnya, aplikasi counter yang menampilkan angka dan bisa bertambah saat tombol ditekan. Perubahan tersebut dikelola menggunakan fungsi setState() yang akan memberi tahu Flutter untuk me-*rebuild* UI dengan nilai terbaru.

Contoh StatefulWidget - Counter App:

```
class MyCounterApp extends StatefulWidget {
  @override
  _MyCounterAppState createState() => _MyCounterAppState();
}

class _MyCounterAppState extends State<MyCounterApp> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Counter App - Stateful'),
      ),
      body: Center(
        child:      Text('Counter:      $_counter',      style:
TextStyle(fontSize: 24)),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        child: Icon(Icons.add),
      ),
    );
  }
}
```

5.2 Komponen UI Interaktif

5.2.1 Dialog

Dialog merupakan widget pop-up kecil yang muncul di atas konten utama aplikasi untuk menyampaikan pesan, peringatan, atau permintaan konfirmasi dari pengguna. Flutter menyediakan berbagai jenis dialog seperti AlertDialog, SimpleDialog, dan Dialog biasa. Umumnya dialog digunakan untuk menampilkan informasi penting atau meminta aksi dari pengguna, seperti memilih "OK" atau "Cancel".

Contoh AlertDialog:

```
void _showDialog(BuildContext context) {
```

```
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        title: Text('Alert'),
        content: Text('This is an alert dialog.'),
        actions: <Widget>[
          TextButton(
            child: Text('OK'),
            onPressed: () {
              Navigator.of(context).pop();
            },
          ),
        ],
      );
    },
  );
}
```

5.2.2 SnackBar

SnackBar digunakan untuk menampilkan notifikasi singkat di bagian bawah layar. Biasanya digunakan untuk memberi tahu pengguna bahwa suatu aksi berhasil atau gagal, tanpa mengganggu tampilan utama aplikasi. SnackBar bersifat sementara dan menghilang secara otomatis setelah beberapa detik. Untuk menampilkannya, digunakan ScaffoldMessenger.of(context).showSnackBar().

Contoh SnackBar:

```
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text('This is a SnackBar'),
    duration: Duration(seconds: 2),
  ),
);
```

5.2.3 TextField

TextField merupakan widget yang digunakan untuk menerima input teks dari pengguna. Widget ini sangat umum dipakai dalam formulir, pencarian, dan login. TextField mendukung berbagai dekorasi dan kontrol, seperti placeholder, ikon, batasan input, serta mengelola nilai input menggunakan TextEditingController.

Contoh TextField:

```
class MyTextField extends StatelessWidget {
```

```
  final
TextEditingController _controller=TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('TextField Example'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: TextField(
          controller: _controller,
          decoration: InputDecoration(
            labelText: 'Enter text',
            border: OutlineInputBorder(),
          ),
        ),
      ),
    );
  }
}
```

5.2.4 DropDown

DropDown atau DropdownButton memungkinkan pengguna memilih salah satu dari beberapa opsi yang tersedia dalam bentuk menu drop-down. Cocok digunakan untuk pemilihan kategori, pilihan kota, jenis kelamin, atau data lainnya yang bersifat pilihan tunggal. Perubahan nilai akan memicu *rebuild* untuk menyesuaikan tampilan yang sedang dipilih.

Contoh DropDown:

```
class MyDropDown extends StatefulWidget {
  @override
  _MyDropDownState createState() => _MyDropDownState();
}

class _MyDropDownState extends State<MyDropDown> {
  String? _selectedItem = 'One';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Dropdown Example')),
```

```
      body: Center(
        child: DropdownButton<String>(
          value: _selectedItem,
          items:<String>['One','Two','Three'].map((String value)
{
            return DropdownMenuItem<String>(
              value: value,
              child: Text(value),
            );
          }).toList(),
          onChanged: (String? newValue) {
            setState(() {
              _selectedItem = newValue;
            });
          },
        ),
      ),
    );
  }
}
```

5.2.5 TabBar

TabBar digunakan untuk menampilkan navigasi horizontal berbasis tab. Setiap tab bisa mewakili tampilan berbeda dalam aplikasi dan dikaitkan dengan TabBarView. Biasanya digunakan untuk mengorganisasi konten dalam kategori atau bagian berbeda, seperti halaman "Beranda", "Profil", dan "Pengaturan". DefaultTabController sering digunakan untuk mengatur jumlah tab dan logika berpindah antar tab.

Contoh TabBar dengan TabBarView:

```
class MyTabBarApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 3,
      child: Scaffold(
        appBar: AppBar(
          title: Text('TabBar Example'),
          bottom: TabBar(
            tabs: [
              Tab(icon: Icon(Icons.directions_car), text: 'Car'),
              Tab(icon: Icon(Icons.directions_bike), text: 'Bike'),
              Tab(icon: Icon(Icons.directions_boat), text: 'Boat'),
```

```
            ],
          ),
        ),
        body: TabBarView(
          children: [
            Center(child: Text('Car Tab')),
            Center(child: Text('Bike Tab')),
            Center(child: Text('Boat Tab')),
          ],
        ),
      ),
    );
  }
}
```

5.3 Navigasi Antar Halaman

5.3.1 BottonNavigationBar

BottomNavigationBar adalah menu navigasi yang berada di bagian bawah layar dan memungkinkan pengguna berpindah antar halaman utama aplikasi dengan cepat. Setiap item pada bar ini biasanya diwakili oleh ikon dan label, seperti “Home”, “Search”, atau “Profile”. Saat salah satu item dipilih, tampilan utama aplikasi akan berubah sesuai index yang diklik.

Contoh BottomNavigationBar:

```
class MyBottomNavBar extends StatefulWidget {
  @override
  _MyBottomNavBarState createState() => _MyBottomNavBarState();
}

class _MyBottomNavBarState extends State<MyBottomNavBar> {
  int _currentIndex = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('BottomNavigationBar Example')),
      body: Center(
        child: Text('Selected index: $_currentIndex'),
      ),
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _currentIndex,
        items: [
```

```
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
          BottomNavigationBarItem(icon: Icon(Icons.search), label: 'Search'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profile'),
        ],
        onTap: (index) {
          setState(() {
            _currentIndex = index;
          });
        },
      ),
    );
  }
}
```

5.3.2 BottomSheet

BottomSheet adalah panel geser dari bagian bawah layar yang menampilkan konten tambahan atau tindakan pengguna. Terdapat dua jenis BottomSheet: modal (menutup interaksi layar utama) dan persistent (tetap berada di layar). Biasanya digunakan untuk menampilkan opsi tindakan cepat atau informasi tambahan, seperti menu berbagi atau filter data.

Contoh BottomSheet:

```
void _showBottomSheet(BuildContext context) {
  showModalBottomSheet(
    context: context,
    builder: (BuildContext context) {
      return Container(
        padding: EdgeInsets.all(16),
        height: 200,
        child: Column(
          children: [
            Text('This is a Bottom Sheet'),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context);
              },
              child: Text('Close'),
            ),
          ],
        ),
      );
    },
  );
```

```
}
```

5.3.3 Drawer

Drawer adalah panel navigasi yang muncul dari sisi kiri (atau kanan) layar, biasanya dibuka dengan menggeser atau ikon hamburger di AppBar. Drawer digunakan untuk navigasi utama aplikasi dan dapat berisi daftar menu, akun pengguna, dan item navigasi penting lainnya. Komponen ini sangat berguna dalam aplikasi dengan banyak fitur atau halaman.

Contoh Drawer:

```
class MyDrawerApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Drawer Example')),
      drawer: Drawer(
        child: ListView(
          padding: EdgeInsets.zero,
          children: <Widget>[
            DrawerHeader(
              decoration: BoxDecoration(color: Colors.blue),
              child:      Text('Drawer      Header',      style:
TextStyle(color: Colors.white, fontSize: 24)),
            ),
            ListTile(
              title: Text('Item 1'),
              onTap: () {
                Navigator.pop(context);
              },
            ),
            ListTile(
              title: Text('Item 2'),
              onTap: () {
                Navigator.pop(context);
              },
            ),
          ],
        ),
      ),
      body: Center(child: Text('Swipe from the left to open the
drawer')),
    );
  }
```

```
}
```

5.3.4 Navigator

Navigator adalah sistem navigasi bawaan Flutter yang digunakan untuk mengelola perpindahan antar halaman (*routes*). Navigator bekerja dengan menambahkan dan menghapus halaman dari *stack*. Metode seperti Navigator.push() digunakan untuk pindah ke halaman baru, sedangkan Navigator.pop() untuk kembali. Flutter juga mendukung *named routes* untuk navigasi yang lebih terstruktur.

Navigasi Dasar dengan Navigator.push:

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => SecondPage()),
);
```

Navigasi dengan Named Routes:

```
Navigator.pushNamed(context, '/secondPage');
```

5.4 Ringkasan dan Kesimpulan

- Stateless dan Stateful Widget merupakan dasar dari pembuatan antarmuka di Flutter. Pemahaman perbedaan keduanya sangat penting untuk menentukan jenis widget yang sesuai dengan kebutuhan interaksi pengguna.
- Komponen UI interaktif seperti *Dialog*, *SnackBar*, *TextField*, *DropDown*, dan *TabBar* memberikan pengalaman pengguna yang dinamis dan responsif. Komponen-komponen ini mempermudah pengambilan input, pemberian notifikasi, dan navigasi antar kategori konten.
- Navigasi dalam aplikasi Flutter tidak hanya sebatas perpindahan halaman menggunakan *Navigator*, tetapi juga mencakup elemen navigasi visual seperti *BottomNavigationBar*, *Drawer*, dan *BottomSheet* yang memperkaya antarmuka pengguna dan membuat aplikasi lebih mudah digunakan.
- Pengelolaan state dan navigasi merupakan fondasi penting dalam membangun aplikasi yang kompleks dan interaktif. Dengan memahami berbagai komponen ini, pengembang dapat menciptakan aplikasi Flutter yang efisien, mudah digunakan, dan memiliki struktur navigasi yang jelas.

CHAPTER 6
CRUD SQLITE

6.1 Konsep dasar Sqflite

Sqflite adalah plugin Flutter yang digunakan untuk mengelola database lokal dengan menggunakan SQLite. SQLite sendiri merupakan sistem manajemen basis data relasional yang ringan dan banyak digunakan untuk menyimpan data secara lokal di perangkat pengguna, tanpa perlu koneksi ke server eksternal.

Dengan Sqflite, pengembang dapat melakukan operasi dasar basis data, seperti:

- Membuat, membaca, memperbarui, dan menghapus (CRUD) data dalam tabel.
- Menyimpan data secara offline, sehingga aplikasi tetap dapat diakses meskipun tanpa koneksi internet.
- Mengelola struktur dan isi database lokal secara efisien.

Beberapa fitur utama yang didukung oleh Sqflite antara lain:

- Eksekusi query SQL standar, seperti SELECT, INSERT, UPDATE, dan DELETE.
- Transaksi database, untuk memastikan data tetap konsisten saat melakukan operasi yang kompleks.
- Pengaturan versi database, memungkinkan migrasi skema saat struktur database mengalami perubahan.

Sqflite sangat bermanfaat dalam pembuatan aplikasi yang membutuhkan penyimpanan data lokal, seperti:

- Aplikasi catatan pribadi,
- Aplikasi daftar tugas (to-do list),
- Aplikasi e-commerce yang mendukung mode offline,
- Dan berbagai aplikasi lain yang memerlukan cache data pengguna.

6.2 Implementasi CRUD menggunakan Sqflite

6.2.1 Setup Dependencies

Ada beberapa cara untuk menambahkan sqflite kedalam project flutter :

- Cara pertama, tambahkan dependensi untuk sqflite dan path di file pubspec.yaml:

```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.0.0+4
  path: ^1.8.1
```

Jalankan perintah berikut di terminal untuk menginstall dependencies:

```
flutter pub get
```

- Cara kedua, jalankan perintah berikut di terminal :

```
flutter pub add sqflite
flutter pub add path
```

Perintah ini akan menambahkan baris ke file pubspec.yaml dan menjalankan flutter pub get secara implisit.

6.2.2 Struktur Folder Project

Untuk memisahkan logika dan antarmuka agar kode lebih rapi dan mudah dikelola, kita perlu membuat beberapa folder di dalam folder lib. Berdasarkan gambar struktur proyek di atas, berikut adalah penjelasannya:

- lib/

  Folder utama tempat semua source code aplikasi Flutter berada.

  - helpers/

    Folder ini digunakan untuk menyimpan file helper, salah satunya db_helper.dart yang berisi logika pengelolaan database seperti koneksi, pembuatan tabel, dan fungsi CRUD.

  - models/

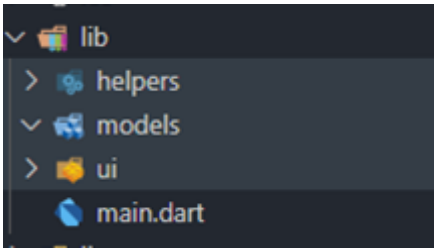
    Berisi file model data, seperti note_model.dart, yang merepresentasikan struktur data yang digunakan dalam aplikasi, termasuk fungsi konversi dari dan ke Map.

  - ui/

    Folder ini menyimpan bagian antarmuka pengguna atau tampilan (User Interface), seperti notes_page.dart.

  - main.dart

    File utama aplikasi Flutter. Di sini biasanya terdapat runApp() yang memanggil widget utama dari aplikasi.



Struktur ini memudahkan pemisahan logika bisnis, data, dan tampilan dalam proyek sehingga lebih modular dan maintainable.

6.2.3 Membuat Model Note

Buat file bernama note_model.dartuntuk model catatan (note):

```
class Note { int? id; String title;
      String description;
      Note({this.id,      required      this.title,      required
  this.description});
      // Convert a Note into a Map. Map<String, dynamic> toMap()
  {
    return { 'id': id,
          'title': title, 'description': description,
        };
      }
      // Convert a Map into a Note.
  static Note fromMap(Map<String, dynamic> map) { return Note(
          id: map['id'],
          title: map['title'], description: map['description'],
        );
      }
    }
```

Kode pada note_model.dart berfungsi sebagai model data untuk objek Note, yang merepresentasikan catatan (note) dalam aplikasi. Model ini membantu dalam memetakan data dari database SQLite ke dalam bentuk objek Dart, dan sebaliknya. Berikut adalah penjelasan dari setiap bagian kode di file tersebut:

a. Deklarasi Kelas Note

```
class Note { int? id; String title;
      String description;
      }
```

`int? id`	Ini adalah properti opsional yang menyimpan ID dari catatan. Pada database, ID biasanya digunakan sebagai primary key yang unik untuk membedakan setiap catatan. Tanda tanya (?) menunjukkan bahwa ID bersifat opsional karena catatan baru tidak memiliki ID sampai disimpan ke database (di mana ID akan otomatis di-generate).
`String title`	Properti ini menyimpan judul catatan.

`String description`	Properti ini menyimpan deskripsi dari catatan.

b. Konstruktor

`Note({this.id,required this.title,required this.description});`	
Konstruktor	Digunakan untuk membuat objek Note. Konstruktor ini menerima tiga parameter
`id`	Opsional, karena belum tentu catatan baru memiliki ID sebelum disimpan.
`Title` dan `description`	Harus diisi (required) karena setiap catatan memerlukan judul dan deskripsi.

c. toMap() Method

```
Map<String, dynamic> toMap() { return {
            'id': id, 'title': title,
            'description': description,
          };
          }
```

`toMap(`	Fungsi ini mengubah objek Note menjadi bentuk Map (peta data), yang merupakan representasi data yang mudah digunakan untuk interaksi dengan database.
Key ('id', 'title', 'description')	Adalah nama kolom di tabel SQLite.
Value (id, title, description)	Adalah nilai dari objek Note yang akan disimpan di database.
Tujuannya adalah agar data Note dapat di-insert atau di-update ke dalam database, yang biasanya memerlukan data dalam bentuk Map<String, dynamic>.	

d. fromMap() Method

```
static Note fromMap(Map<String, dynamic> map) { return
      Note(
            id: map['id'],
            title: map['title'], description:
        map['description'],
          );
          }
```

`fromMap()`	Fungsi ini mengonversi data dari bentuk Map yang diperoleh dari database menjadi objek Note.
id: map['id']	Mengambil nilai ID dari map dan mengisinya ke properti id dari objek Note.
title: map['title'] dan description: map['description']	Mengambil nilai judul dan deskripsi dari map, lalu mengisi properti title dan description pada objek Note.
Tujuan metode ini adalah untuk mengambil data dari database (yang biasanya dalam bentuk Map<String, dynamic>) dan membuatnya menjadi objek Note yang dapat diproses dalam aplikasi.	

File note_model.dartini berfungsi untuk:

1. Mendefinisikan struktur data untuk objek catatan (Note).
2. Menyediakan cara untuk mengonversi objek Note menjadi Map untuk disimpan ke database (melalui toMap()).
3. Mengonversi Map dari database menjadi objek Note yang bisa digunakan dalam kode (melalui fromMap()).

Contoh alur penggunaannya:

1. Saat membuat catatan baru, objek Note diubah menjadi Map menggunakan toMap(), lalu dimasukkan ke database.
2. Saat mengambil data dari database, data tersebut dikonversi kembali menjadi objek Note menggunakan fromMap().

6.2.4 Membuat DBHelper

Buat file bernama db_helper.dart untuk mengatur SQLite:

```
import
'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import '../models/note_model.dart';


class DBHelper {
```

```
     static final DBHelper _instance =
     DBHelper._internal(); factory
     DBHelper()=> _instance;

            static Database?
_database;

     DBHelper._internal();

     Future<Database> get database
     async {

          if (_database != null)
     return_database!;

          _database = await _initDB();
     return _database!;

          }

          Future<Database> _initDB()
     async {

          String path = join(await getDatabasesPath(),
     'notes.db'); return await openDatabase(

               path,

               version: 1,

               onCreate: (db, version) async {

                  await db.execute('''

               CREATE TABLE notes(

               id INTEGER PRIMARY KEY
     AUTOINCREMENT,

               title TEXT, description
     TEXT

                        )

                ''');

                },

            );

          }

          // Insert Notes

          Future<int> insert(Note note) async {

          Database db = await database;

          return await
     db.insert('notes', note.toMap());

          }

          // Get all Notes

          Future<List<Note>> getNotes() async {

          Database db = await database;
```

```
        var result = await
    db.query('notes');

        return result.map((map) =>
    Note.fromMap(map)).toList();

        }

        // Update Note

        Future<int> update(Note note)
    async {

              Database db = await database;

              return await db.update(

                'notes',

                note.toMap(),

                where: 'id = ?',
      whereArgs: [note.id],

              );
            }
       // Delete Note
       Future<int> delete(int id) async {

       Database db = await database; return await db.delete(

                 'notes',

              where: 'id = ?',
      whereArgs: [id],

            );
          }
        }
```

File db_helper.dart bertanggung jawab untuk mengelola operasi database menggunakan SQLite di Flutter. Di sini, kita menggunakan library sqflite untuk menangani interaksi dengan database lokal. DBHelper menyediakan metode CRUD (Create, Read, Update, Delete) untuk tabel catatan (notes). Berikut adalah penjelasan kode di file db_helper.dart:

a) Deklarasi Singleton pada Kelas DBHelper

```
class DBHelper {

      static final DBHelper _instance = DBHelper._internal();

      factory DBHelper() => _instance;

      static Database? _database;

           DBHelper._internal();

        }
```

- DBHelper._internal(): Ini adalah private constructor yang hanya bisa dipanggil dari dalam kelas. Dengan menggunakan pola singleton, kita memastikan bahwa hanya satu instance dari DBHelper yang dibuat selama aplikasi berjalan.
- static final DBHelper _instance: Membuat instance tunggal dari DBHelper untuk digunakan di seluruh aplikasi.
- factory DBHelper(): Menyediakan akses ke instance tunggal DBHelper. Setiap kali DBHelper() dipanggil, instance yang sama akan dikembalikan.
- static Database? _database: Properti ini akan menyimpan referensi ke objek Database setelah terhubung dengan SQLite. Tanda ? menunjukkan bahwa ini bisa null, artinya belum ada instance Database saat kelas pertama kali dipanggil.

b) Membuat atau Mengakses Database

```
Future<Database> get database async {

  if (_database != null) return _database!;

  _database = await _initDB(); return
  _database!;

}
```

- Future<Database>: Metode ini mengembalikan objek Database. Ini akan digunakan untuk mengakses atau membuat database.
- Pengecekan if (_database != null): Jika instance database sudah ada, kembalikan instance tersebut agar tidak membuat koneksi baru ke database.
- _initDB(): Jika belum ada instance database, metode ini dipanggil untuk membuat database baru.

c) Inisialisasi Database

```
Future<Database> _initDB() async {

String path = join(await getDatabasesPath(), 'notes.db'); return
await openDatabase(

path, version: 1,

onCreate: (db, version) async { await
db.execute('''

CREATE TABLE notes(

id INTEGER PRIMARY KEY AUTOINCREMENT,

title TEXT, description TEXT

) ''');

},
);

}
```

- String path = join(await getDatabasesPath(), 'notes.db');: Mendapatkan path direktori penyimpanan database pada perangkat, dan menentukan nama file database (notes.db).
- openDatabase(): Metode ini membuka (atau membuat) database di lokasi path dengan versi yang ditentukan (dalam hal ini versi 1).
    - onCreate(): Callback ini dipanggil ketika database pertama kali dibuat. Di sini, kita mengeksekusi perintah SQL untuk membuat tabel notes dengan kolom:
        - id: Kolom id berfungsi sebagai primary key yang secara otomatis ditingkatkan (autoincrement).
        - title: Kolom untuk menyimpan judul catatan.
        - description: Kolom untuk menyimpan deskripsi catatan.

d) Metode CRUD (Create, Read, Update, Delete)

1) Insert (Create)

```
Future<int> insert(Note note) async {
      Database db = await database;

           return await db.insert('notes', note.toMap());
     }
```

- Database db = await database;: Mendapatkan instance database.
- db.insert('notes', note.toMap());: Menyisipkan data catatan ke tabel notes. Objek note diubah menjadi Map menggunakan metode toMap()

dari model Note. Hasilnya adalah int yang berisi ID dari baris yang baru ditambahkan.

2) Get All Notes (Read)

```
Future<List<Note>> getNotes() async { Database
      db = await database;
           var result = await db.query('notes');
           return result.map((map) =>Note.fromMap(map)).toList();
     }
```

- db.query('notes');: Mengambil semua catatan dari tabel notes dalam bentuk list Map<String, dynamic>.
- .map((map) => Note.fromMap(map)).toList();: Setiap elemen dari list tersebut dikonversi dari Map menjadi objek Note menggunakan metode fromMap(), kemudian hasilnya dikembalikan dalam bentuk list List<Note>.

3) Update

```
Future<int> update(Note note) async {
      Database db = await database; return
      await db.update(
                'notes',
                note.toMap(),
                where: 'id = ?',
           whereArgs: [note.id],
            );
     }
```

- db.update('notes', note.toMap(), where: 'id = ?', whereArgs: [note.id]);: Metode ini memperbarui catatan berdasarkan id. Kita menyertakan klausa WHERE untuk menentukan baris mana yang harus di-update. note.id adalah ID catatan yang akan diperbarui. Objek Note diubah menjadi Map menggunakan toMap().

4) Delete

```
Future<int>  delete(int  id)  async  {
  Database  db  =  await  database;  return
  await db.delete(
       'notes',
       where:  'id  =  ?',  whereArgs:
  [id],
  );
}
```

- db.delete('notes', where: 'id = ?', whereArgs: [id]);: Menghapus catatan berdasarkan id dari tabel notes. Klausa WHERE digunakan untuk menentukan baris mana yang akan dihapus.

File db_helper.dart ini bertanggung jawab untuk mengelola seluruh operasi database (CRUD) di aplikasi. Fungsi utamanya meliputi:

Membuka dan menginisialisasi database.

CRUD:

Create: Menyisipkan catatan baru ke database.

Read: Mengambil semua catatan dari database.

Update: Memperbarui catatan yang ada berdasarkan id.

Delete: Menghapus catatan dari database berdasarkan id.

Singleton digunakan untuk memastikan bahwa hanya ada satu instance dari DBHelper selama aplikasi berjalan, sehingga tidak ada koneksi database ganda yang tidak perlu.

6.2.5 Membuat CRUD UI

Berikut adalah contoh halaman untuk menampilkan, menambahkan, mengedit, dan menghapus catatan.

```
import 'package:flutter/material.dart';
import '../helpers/db_helper.dart';
import '../models/note_model.dart';

class NotesPage extends StatefulWidget {
  @override
  _NotesPageState createState() => _NotesPageState();
}

class _NotesPageState extends State<NotesPage> {
  List<Note> notes = [];
  TextEditingController titleController = TextEditingController();
```

```
  TextEditingController          descriptionController          =
TextEditingController();

  @override
  void initState() {
    super.initState();
    _loadNotes();
  }

  void _loadNotes() async {
    List<Note> list = await DBHelper().getNotes();
    setState(() {
      notes = list;
    });
  }

  void _saveNote() async {
    String title = titleController.text;
    String description = descriptionController.text;
    if (title.isNotEmpty && description.isNotEmpty) {
      Note note = Note(
        title: title,
        description: description,
      );
      await DBHelper().insert(note);
      _loadNotes();
      titleController.clear();
      descriptionController.clear();
    }
  }

  void _editNote(Note note) {
    titleController.text = note.title;
    descriptionController.text = note.description;

    showDialog(
      context: context,
      builder: (_) => AlertDialog(
        title: Text('Edit Note'),
        content: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            TextField(
              controller: titleController,
              decoration: InputDecoration(labelText: 'Title'),
            ),
            TextField(
              controller: descriptionController,
              decoration:InputDecoration(labelText: Description'),
            ),
          ],
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.pop(context);
            },
            child: Text('Cancel'),
          ),
          TextButton(
```

```
            onPressed: () async {
              note.title = titleController.text;
              note.description = descriptionController.text;
              await DBHelper().update(note);
              _loadNotes();
              titleController.clear();
              descriptionController.clear();
              Navigator.pop(context);
            },
            child: Text('Save'),
          ),
        ],
      ),
    );
  }

  void _deleteNote(int id) async {
    await DBHelper().delete(id);
    _loadNotes();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Notes'),
      ),
      body: Column(
        children: [
          Padding(
            padding: EdgeInsets.all(8.0),
            child: TextField(
              controller: titleController,
              decoration: InputDecoration(labelText: 'Title'),
            ),
          ),
          Padding(
            padding: EdgeInsets.all(8.0),
            child: TextField(
              controller: descriptionController,
              decoration:                InputDecoration(labelText:
'Description'),
            ),
          ),
          ElevatedButton(
            onPressed: _saveNote,
            child: Text('Save Note'),
          ),
          Expanded(
            child: ListView.builder(
              itemCount: notes.length,
              itemBuilder: (context, index) {
                Note note = notes[index];
                return ListTile(
                  title: Text(note.title),
                  subtitle: Text(note.description),
                  trailing: Row(
                    mainAxisSize: MainAxisSize.min,
                    children: [
                      IconButton(
```

```
                        icon: Icon(Icons.edit),
                        onPressed: () => _editNote(note),
                      ),
                      IconButton(
                        icon: Icon(Icons.delete),
                        onPressed: () => _deleteNote(note.id!),
                      ),
                    ],
                  ),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}
```

Kode pada notes_page.dart bertanggung jawab untuk menampilkan antarmuka pengguna (UI) dan menangani logika CRUD (Create, Read, Update, Delete) dari catatan yang disimpan di SQLite. Pada file ini, kita membuat halaman utama yang menampilkan daftar catatan, serta fitur untuk menambah, mengedit, dan menghapus catatan. Berikut adalah penjelasan dari setiap bagian kode di file notes_page.dart:

a) Deklarasi dan Inisialisasi NotesPage

```
class NotesPage extends StatefulWidget { @override
      _NotesPageState createState() => _NotesPageState();
    }
```

- NotesPage: Merupakan widget yang menggunakan StatefulWidget, karena halaman ini membutuhkan state yang dapat berubah, seperti menambah, menghapus, dan mengedit catatan.
- _NotesPageState: Adalah class yang berfungsi untuk mengelola logika dan state dari halaman NotesPage.

b) Deklarasi Variabel State

```
List<Note notes = [];
TextEditingController titleController = TextEditingController();
 TextEditingController descriptionController =  TextEditingController();
```

- List<Note> notes: Ini adalah list yang digunakan untuk menyimpan catatan yang akan ditampilkan dalam UI.
- TextEditingController titleController:Controller untuk menangani input teks pada field judul catatan.

- TextEditingController descriptionController: Controller untuk menangani input teks pada field deskripsi catatan.

c) Inisialisasi Data dengan _loadNotes()

```
@override
 void initState() {
   super.initState();
   _loadNotes();
     }
```

- initState(): Fungsi ini dipanggil ketika halaman pertama kali dibangun. Di sini kita memanggil fungsi _loadNotes() untuk memuat catatan dari database SQLite ketika halaman dibuka.

d) Memuat Data dari Database (_loadNotes())

```
void _loadNotes() async {
     List<Note> list = await DBHelper().getNotes();
     setState(() {
          notes = list;
        });
      }
```

- _loadNotes(): Fungsi ini mengambil data catatan dari database menggunakan metode getNotes() dari DBHelper.
- setState(): Setelah catatan diambil, kita memanggil setState() untuk memperbarui UI dengan daftar catatan yang baru dimuat. Dengan begitu, aplikasi akan merender ulang tampilan dengan catatan yang diperbarui.

e) Menyimpan Catatan Baru (_saveNote())

```
void _saveNote() async {
    String title = titleController.text;
    String description = descriptionController.text;

    if (title.isNotEmpty && description.isNotEmpty) {
    Note note = Note(
      title:             title,
     description: description,
    );
    await DBHelper().insert(note);
    _loadNotes();
    titleController.clear();
    descriptionController.clear();
  }
}
```

- _saveNote(): Fungsi ini menyimpan catatan baru yang diinput pengguna.
    - **Validasi Input**: Jika judul dan deskripsi tidak kosong, kita membuat objek Note baru.
    - DBHelper().insert(note): Metode ini menyimpan catatan ke database.
    - _loadNotes(): Setelah menyimpan catatan baru, data diperbarui dengan memanggil ulang fungsi untuk memuat catatan.
    - clear(): Membersihkan teks input setelah catatan disimpan.

f) Mengedit Catatan (_editNote())

```
void _editNote(Note note) {
  titleController.text = note.title;
  descriptionController.text = note.description;

  showDialog( context:
    context,
    builder: (_) => AlertDialog(
     title: Text('Edit Note'),
     content: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              TextField(
                controller: titleController,
                decoration: InputDecoration(labelText: 'Title'),
              ),
              TextField(
                controller: descriptionController,
                decoration: InputDecoration(labelText: 'Description'),
              ),
            ],
          ),
      actions: [
        TextButton(
          onPressed: () {
            Navigator.pop(context);
              },
              child: Text('Cancel'),
            ),
            TextButton(
              onPressed: () async {
                note.title = titleController.text;
                note.description = descriptionController.text;
                await DBHelper().update(note);
                _loadNotes();
                titleController.clear();
                descriptionController.clear();
                Navigator.pop(context);
              },
              child: Text('Save'),
            ),
          ],
        ),
      );
    }
```

- **_editNote()**: Fungsi ini membuka dialog untuk mengedit catatan yang dipilih.
    - **showDialog()**: Menampilkan dialog yang memungkinkan pengguna untuk mengedit judul dan deskripsi catatan.
    - **TextField**: Menampilkan input teks untuk judul dan deskripsi yang diisi dengan nilai dari catatan yang akan diedit.

- **Tombol "Save"**: Setelah pengguna selesai mengedit, perubahan disimpan ke database menggunakan DBHelper().update(note), dan daftar catatan diperbarui.
- **Tombol "Cancel"**: Menutup dialog tanpa menyimpan perubahan.

g) Menghapus Catatan (_deleteNote())

```
void _deleteNote(int id) async{
  await DBHelper().delete(id);
       _loadNotes();
    }
```

- **_deleteNote()**: Fungsi ini menghapus catatan berdasarkan id dari database menggunakan metode delete() dari DBHelper. Setelah itu, daftar catatan dimuat ulang untuk memperbarui UI.

h) Membangun Tampilan UI (build)

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Notes'),
    ),
    body: Column(
      children: [
        Padding(
          padding: EdgeInsets.all(8.0),
          child: TextField(
            controller: titleController,
            decoration: InputDecoration(labelText: 'Title'),
          ),
        ),
        Padding(
          padding: EdgeInsets.all(8.0),
          child: TextField(
            controller: descriptionController,
            decoration:              InputDecoration(labelText:
'Description'),
          ),
        ),
        ElevatedButton(
          onPressed: _saveNote,
          child: Text('Save Note'),
        ),
        Expanded(
          child: ListView.builder(
            itemCount: notes.length,
            itemBuilder: (context, index) {
              Note note = notes[index];
              return ListTile(
                title: Text(note.title),
                subtitle: Text(note.description),
                trailing: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    IconButton(
```

```
                        icon: Icon(Icons.edit),
                        onPressed: () => _editNote(note),
                      ),
                      IconButton(
                        icon: Icon(Icons.delete),
                        onPressed: () => _deleteNote(note.id!),
                      ),
                    ],
                  ),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
```

- **Scaffold**: Mengatur struktur dasar halaman dengan AppBar dan body.
    - **AppBar**: Menampilkan judul halaman ("Notes").
    - **TextField untuk Input**: Dua input teks (untuk judul dan deskripsi catatan) di atas daftar catatan.
    - **ElevatedButton**: Tombol untuk menyimpan catatan baru. Ketika ditekan, fungsi _saveNote() akan dipanggil.
    - **ListView.builder**: Digunakan untuk menampilkan daftar catatan yang diambil dari database. Setiap catatan ditampilkan dalam bentuk **ListTile**.
        - **ListTile**: Menampilkan judul dan deskripsi catatan.
        - **Trailing Actions**: Menambahkan dua tombol di sebelah kanan tiap catatan:
            - Tombol **edit** untuk mengedit catatan.
            - Tombol **delete** untuk menghapus catatan.

File notes_page.dart ini bertanggung jawab untuk:

1. **Menampilkan UI** yang berisi daftar catatan dan input untuk menambah catatan baru.
2. Menyediakan fungsi untuk **CRUD** (Create, Read, Update, Delete) dengan operasi database yang dilakukan menggunakan DBHelper.
3. Memperbarui tampilan secara dinamis ketika data diubah (misalnya, ketika catatan ditambahkan, dihapus, atau diubah).

6.2.6 Menjalankan main.dart

Terakhir, buat file main.dart:

```
import 'package:flutter/material.dart';
import '../ui/notes_page.dart';


void main() { runApp(MyApp());
}


class MyApp extends StatelessWidget { @override
  Widget build(BuildContext context) { return
    MaterialApp(
      title: 'Flutter SQLite CRUD', theme:
      ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: NotesPage(),
    );
  }
}
```

CHAPTER 7
PENGELOLAAN MEDIA DALAM APLIKASI

7.1 Pengelolaan Gambar

Flutter menyediakan berbagai paket untuk menangani gambar. Dapat menggunakan pustaka seperti image_picker untuk memilih gambar dari galeri atau kamera, dan cached_network_image untuk menampilkan gambar dari URL dengan caching.

- Langkah 1: Tambahkan Dependensi

  Pertama, tambahkan dependensi image_picker pada file pubspec.yaml

- Langkah 2: Mengatur Izin

  Pastikan sudah menambahkan izin yang diperlukan untuk mengambil gambar dari kamera atau galeri pada platform Android dan iOS.

  Android: Tambahkan izin pada file `android/app/src/main/AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission
android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

  iOS: Tambahkan deskripsi izin dalam file `ios/Runner/Info.plist`:

```
<key>NSPhotoLibraryUsageDescription</key>
<string>We need access to your photo library to pick images.</string>
<key>NSCameraUsageDescription</key>
<string>We need access to your camera to take photos.</string>
```

- Langkah 3: Kode Flutter

  Berikut adalah kode untuk menggunakan image_picker untuk memilih gambar dari galeri atau kamera dan menampilkannya dalam aplikasi Flutter.

```
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'dart:io';

class ImagePickerExample extends StatefulWidget {
  @override
  _ImagePickerExampleState createState() =>
_ImagePickerExampleState();
}

class _ImagePickerExampleState extends State<ImagePickerExample> {
  File? _image; // Untuk menyimpan gambar yang dipilih

  final ImagePicker _picker = ImagePicker();

  // Fungsi untuk memilih gambar dari galeri
  Future<void> _pickImageFromGallery() async {
```

```
    final XFile? image = await _picker.pickImage(source:
ImageSource.gallery);
    if (image != null) {
      setState(() {
        _image = File(image.path);
      });
    }
  }

  // Fungsi untuk mengambil gambar dengan kamera
  Future<void> _pickImageFromCamera() async {
    final XFile? image = await _picker.pickImage(source:
ImageSource.camera);
    if (image != null) {
      setState(() {
        _image = File(image.path);
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Image Picker Example'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            _image != null
                ? Image.file(
                    _image!,
                    width: 200,
                    fit: BoxFit.cover,
                  )
                : Text('No image selected.'),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: _pickImageFromGallery,
              child: Text('Pick Image from Gallery'),
            ),
            SizedBox(height: 10),
            ElevatedButton(
              onPressed: _pickImageFromCamera,
              child: Text('Take Photo with Camera'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Penjelasan Kode:

1. Stateful Widget: ImagePickerExample adalah StatefulWidget karena kita perlu mengubah state aplikasi ketika gambar dipilih.
2. Mengambil Gambar:
   - _pickImageFromGallery() menggunakan ImagePicker.pickImage() untuk memilih gambar dari galeri.
   - _pickImageFromCamera() menggunakan ImagePicker.pickImage() untuk mengambil gambar dengan kamera.
3. Menampilkan Gambar: Jika gambar berhasil dipilih, gambar akan ditampilkan menggunakan widget Image.file. Jika belum ada gambar yang dipilih, aplikasi akan menampilkan teks "No image selected."
4. Tombol untuk Memilih Gambar: Ada dua tombol untuk memilih gambar—satu untuk galeri dan satu lagi untuk kamera.

7.2 Pengelolaan Video

Untuk video, bisa menggunakan paket video_player untuk memutar video di dalam aplikasi. Paket ini mendukung pemutaran video dari URL atau file lokal.

- Langkah 1: Tambahkan Dependensi

  Tambahkan dependensi video_player dan image_picker ke dalam pubspec.yaml

- Langkah 2: Izin untuk Akses Penyimpanan

  Pastikan menambahkan izin yang diperlukan untuk perangkat Android dan iOS.

  Android: Tambahkan izin dalam file android/app/src/main/AndroidManifest.xml.

```
<uses-permission
android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.CAMERA"/>
```

  iOS: Tambahkan deskripsi izin pada file ios/Runner/Info.plist.

```
<key>NSPhotoLibraryUsageDescription</key>
<string>We need access to your photo library to pick
videos.</string>
<key>NSCameraUsageDescription</key>
<string>We need access to your camera to take videos.</string>
```

- Langkah 3: Kode Flutter

  Berikut adalah kode untuk memilih video menggunakan image_picker dan memutar video tersebut menggunakan video_player.

```
import 'package:flutter/material.dart';
```

```
import 'package:image_picker/image_picker.dart';
import 'package:video_player/video_player.dart';
import 'dart:io';

class VideoPlayerExample extends StatefulWidget {
      @override
      _VideoPlayerExampleState createState() =>
_VideoPlayerExampleState();
    }

class _VideoPlayerExampleState extends State<VideoPlayerExample> {
      late VideoPlayerController _controller;
      File? _videoFile;
      bool _isPlaying = false;

      final ImagePicker _picker = ImagePicker();

      // Fungsi untuk memilih video dari galeri atau penyimpanan
      Future<void> _pickVideo() async {
        final XFile? video = await _picker.pickVideo(source:
ImageSource.gallery);
        if (video != null) {
          setState(() {
            _videoFile = File(video.path);
          });
          _initializeVideo();
        }
      }

      // Inisialisasi controller video
      void _initializeVideo() {
        _controller = VideoPlayerController.file(_videoFile!)
          ..initialize().then((_) {
            setState(() {});
          });
      }

      // Fungsi untuk play/pause video
      void _togglePlayPause() {
        setState(() {
          if (_isPlaying) {
            _controller.pause();
          } else {
            _controller.play();
          }
          _isPlaying = !_isPlaying;
        });
      }

      @override
      void dispose() {
        super.dispose();
        _controller.dispose();
    // Hapus controller saat widget dihancurkan
      }

      @override
      Widget build(BuildContext context) {
        return Scaffold(
          appBar: AppBar(
```

```
          title: Text('Video Picker and Player'),
        ),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [

  // Menampilkan video jika sudah dipilih dan diinisialisasi
              _videoFile != null && _controller.value.isInitialized
                  ? Column(
                      children: [
                        AspectRatio(
                          aspectRatio: _controller.value.aspectRatio,
                          child: VideoPlayer(_controller),
                        ),
                        IconButton(
                          icon: Icon(
                           _isPlaying ? Icons.pause : Icons.play_arrow,
                            size: 50,
                          ),
                          onPressed: _togglePlayPause,
                        ),
                      ],
                    )
                  : Text('No video selected'),
              SizedBox(height: 20),
              ElevatedButton(
                onPressed: _pickVideo, // Tombol untuk memilih video
                child: Text('Pick Video'),
              ),
            ],
          ),
        ),
      );
    }
  }
```

Penjelasan Kode:

1. Menggunakan ImagePicker untuk Memilih Video:
   - Menggunakan ImagePicker().pickVideo() untuk memilih video dari galeri. Anda dapat mengganti ImageSource.gallery dengan ImageSource.camera jika ingin memilih video dari kamera.
   - Setelah video dipilih, file video disimpan dalam variabel _videoFile.
2. Menyiapkan Video dengan video_player:
   - Setelah memilih video, kita membuat VideoPlayerController yang diinisialisasi menggunakan file yang dipilih (_videoFile).
   - VideoPlayerController.file digunakan untuk memutar video dari file lokal.
3. Memutar atau Menjeda Video:
   - Tombol play/pause mengontrol pemutaran video. Saat video diputar, tombol berubah menjadi ikon jeda dan sebaliknya.

4. Menampilkan Video:
   - Video ditampilkan menggunakan widget AspectRatio untuk menjaga rasio aspek video yang benar. VideoPlayer(_controller) digunakan untuk memutar video yang terhubung dengan controller.
5. Dispose:
   - Pada metode dispose, kita memastikan untuk membuang controller video ketika widget dihancurkan untuk menghindari kebocoran memori.

7.3 Pengelolaan Audio

Untuk audio, dapat menggunakan paket audioplayers untuk memutar file audio dari URL atau file lokal.

- Langkah 1: Tambahkan Dependensi

  Tambahkan dependensi audioplayers pada file pubspec.yaml.
- Langkah 2: Izin untuk Akses Penyimpanan (Jika Menggunakan File Lokal)

  Jika ingin mengakses file audio lokal, pastikan memiliki izin untuk mengakses penyimpanan perangkat, terutama di Android.

Android (untuk mengakses file lokal):

Tambahkan izin pada file `android/app/src/main/AndroidManifest.xml:

<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>

<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>

iOS: Tidak ada izin khusus yang diperlukan untuk memutar file audio, tetapi jika mengakses jaringan, pastikan menangani izin jaringan.

- **Langkah 3: Kode Flutter**

  Berikut adalah contoh kode untuk memutar audio menggunakan audioplayers.

```
import 'package:flutter/material.dart';
import 'package:audioplayers/audioplayers.dart';

class AudioPlayerExample extends StatefulWidget {
  @override
  _AudioPlayerExampleState createState() => _AudioPlayerExampleState();
}

class _AudioPlayerExampleState extends State<AudioPlayerExample> {
  late AudioPlayer _audioPlayer;
  bool _isPlaying = false;

  @override
  void initState() {
    super.initState();
```

```
    _audioPlayer = AudioPlayer(); // Inisialisasi AudioPlayer
  }

  // Fungsi untuk memutar audio
  Future<void> _playAudio() async {
    String audioUrl =
'https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3';
    await _audioPlayer.play(UrlSource(audioUrl));
    setState(() {
      _isPlaying = true;
    });
  }

  // Fungsi untuk menghentikan audio
  Future<void> _stopAudio() async {
    await _audioPlayer.stop();
    setState(() {
      _isPlaying = false;
    });
  }

  // Fungsi untuk menjeda audio
  Future<void> _pauseAudio() async {
    await _audioPlayer.pause();
    setState(() {
      _isPlaying = false;
    });
  }

  @override
  void dispose() {
    super.dispose();
    _audioPlayer.dispose(); // Hapus objek audio player saat widget
dihancurkan
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Audio Player Example'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              _isPlaying ? 'Audio is Playing' : 'Audio is Stopped',
              style: TextStyle(fontSize: 20),
            ),
            SizedBox(height: 20),
            IconButton(
              icon: Icon(
                _isPlaying ? Icons.pause : Icons.play_arrow,
                size: 50,
              ),
              onPressed: _isPlaying ? _pauseAudio : _playAudio,
            ),
            SizedBox(height: 20),
            IconButton(
```

```
                icon: Icon(
                  Icons.stop,
                  size: 50,
                ),
                onPressed: _stopAudio,
              ),
            ],
          ),
        ),
      );
    }
}
```

Penjelasan Kode:

1. Menggunakan AudioPlayer:

   AudioPlayer adalah kelas yang digunakan untuk memutar, menjeda, dan menghentikan audio. Anda perlu menginisialisasi objek AudioPlayer di initState().

2. Memutar Audio:

   Fungsi _playAudio akan memutar audio dari URL. Anda dapat mengganti URL ini dengan URL audio Anda sendiri atau mengganti dengan path lokal jika menggunakan file lokal (menggunakan AudioPlayer.play('file:///path/to/audio.mp3')). Anda bisa menggunakan play() dengan URL audio atau file lokal.

3. Menghentikan Audio:

   Fungsi _stopAudio akan menghentikan audio yang sedang diputar.

4. Menjeda Audio:

   Fungsi _pauseAudio akan menjeda audio yang sedang diputar dan mengubah status ke "paused".

5. Menampilkan Status Pemutaran Audio:

   Menggunakan widget IconButton, aplikasi akan menampilkan ikon play, pause, atau stop berdasarkan status audio (_isPlaying).

6. Dispose AudioPlayer:

   Pada metode dispose(), kita membuang objek _audioPlayer agar tidak ada kebocoran memori saat widget dihancurkan.

CHAPTER 8
DEPLOYMENT FLUTTER

8.1 Pengantar Deployment

Deployment adalah proses mempersiapkan, membangun, dan merilis aplikasi agar bisa digunakan oleh pengguna akhir. Setelah aplikasi selesai dikembangkan dan diuji, tahap selanjutnya adalah memastikan aplikasi dapat dijalankan di perangkat target dengan lancar. Di Flutter, proses deployment sangat fleksibel karena mendukung banyak platform dari satu basis kode yang sama.

Platform Tujuan Deployment:

Flutter memungkinkan aplikasi dikembangkan dan di-deploy ke berbagai platform berikut:

- **Android (Google Play Store)**
  Flutter secara native mendukung Android. Aplikasi dapat dibangun dalam format .apk atau .aab (Android App Bundle) untuk distribusi melalui Play Store atau diinstal langsung (sideloading).
- **iOS (Apple App Store)**
  Untuk deployment ke iOS, Flutter menggunakan Xcode dan menghasilkan file .ipa. Proses ini memerlukan akun developer Apple dan hanya bisa dilakukan di macOS.
- **Web**
  Flutter dapat membangun aplikasi web dalam bentuk HTML, CSS, dan JavaScript. Aplikasi web Flutter bisa di-deploy ke hosting seperti Firebase Hosting, GitHub Pages, atau server pribadi.
- **Desktop (Windows, macOS, Linux)**
  Flutter juga mendukung aplikasi desktop dengan tampilan native. Aplikasi bisa dibangun menjadi file .exe (Windows), .app (macOS), atau binary Linux, dan bisa didistribusikan secara langsung atau melalui toko aplikasi desktop seperti Microsoft Store.

8.2 Persiapan Deployment

8.2.1 Instalasi dan Konfigurasi Lingkunga

Sebelum melakukan deployment, pastikan alat dan lingkungan sudah disiapkan:

- **Flutter SDK:** Sudah terpasang dengan benar.

- **Editor:** Gunakan Visual Studio Code atau Android Studio.
- **Android Studio:** Dibutuhkan untuk membangun aplikasi Android.
- **Xcode:** Dibutuhkan untuk membangun aplikasi iOS (khusus macOS).

Cek sistem dengan:

```
flutter doctor
```

Pastikan semua kebutuhan sudah **tercentang hijau**

8.2.2 Menyiapkan Project

1. Mengganti Nama dan Deskripsi Aplikasi

Edit `pubspec.yaml`:

```
name: nama_aplikasi
description: Deskripsi aplikasi Anda
```

2. Mengubah Icon Aplikasi

Gunakan package flutter_launcher_icons.

- Tambahkan ke dev_dependencies:

```
dev_dependencies:
      flutter_launcher_icons: ^0.14.2
```

- Tambahkan konfigurasi:

```
flutter_icons:
     android: true
     ios: true
     image_path: "assets/icon/app_icon.png"
```

- Jalankan perintah:

```
flutter pub get
flutter pub run flutter_launcher_icons
```

8.2.3 Konfigurasi Splash Screen

Untuk menampilkan layar pembuka (splash screen) saat aplikasi Flutter pertama kali dibuka, kamu bisa menggunakan package flutter_native_splash. Splash screen ini biasanya menampilkan logo atau branding aplikasi sebelum halaman utama ditampilkan.

Langkah-langkah:

- Tambahkan dependency ke pubspec.yaml:

Tambahkan dependency:

```
dev_dependencies:
  flutter_native_splash: ^2.4.3
```

- Tambahkan konfigurasi splash screen:

Masukkan konfigurasi di bawah ini di dalam file pubspec.yaml, sesuaikan warna dan path gambar sesuai kebutuhan kamu:

```
flutter_native_splash:
   color: "#FFFFFF"
   image: assets/icon/app_splash.png
```

- Jalankan:

```
flutter pub run flutter_native_splash:create
```

8.3 Deployment Untuk Platform

8.3.1 Android (APK & AAB)

a. Perbedaan APK dan AAB

Tabel 8. 1 Perbedaan APK dan AAB

Aspek	APK (.APK)	AAB (.AAB)
Distribusi	Langsung diinstal di perangkat Android	Hanya bisa melalui Google Play Store
Ukuran File	Lebih besar (semua varian disertakan)	Lebih kecil (dioptimasi berdasarkan perangkat)
Isi File	Semua ABI, resolusi, bahasa	Varian terpisah, Play Store hanya mengambil yang dibutuhkan
Optimasi	Tidak dioptimasi	Dioptimasi otomatis oleh Play Store
Cocok Untuk	Testing, distribusi manual (offline)	Distribusi resmi via Play Store
Instalasi	Manual	Melalui Play Store
Kompleksitas Build	Lebih sederhana	Lebih kompleks, perlu setup signing dan Play Console

b. Langkah Deployment Android

1) Buat Keystore

Buat filw keystore.jks untuk menandatangani aplikasi:

```
keytool -genkey -v -keystore keystore.jks -keyalg RSA -keysize 2048 -validity 36500 -alias key_alias
```

- Simpan file keystore.jks ke dalam folder android/
- Gantilah key_alias dan password sesuai keinginan Anda

2) Buat File key.properties

Buat file key.properties di folder android/:

```
storePassword=your_password
```

```
keyPassword=your_password
keyAlias=key_alias
storeFile=../keystore.jks
```

3) Konfigurasi build.gradle

Edit file android/app/build.gradle:

```
def keystoreProperties = new Properties()
def keystorePropertiesFile =
rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new
FileInputStream(keystorePropertiesFile))
}

android {
    // Konfigurasi lainnya seperti compileSdkVersion
dsb.

    signingConfigs {
        release {
            storeFile
file(keystoreProperties['storeFile'])
            storePassword
keystoreProperties['storePassword']
            keyAlias keystoreProperties['keyAlias']
            keyPassword
keystoreProperties['keyPassword']
        }
    }

    buildTypes {
        release {
            signingConfig signingConfigs.release
            minifyEnabled true
            shrinkResources true
            proguardFiles
getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
        }
    }
}
```

4) Build APK/AAB

```
flutter build apk --release
flutter build appbundle --release
```

File akan berada di:

```
build/app/outputs/flutter-apk/
```

5) Upload ke Google Play Console

- Daftar sebagai Google Play Developer
- Buat aplikasi baru dan upload file .aab di bagian "Release > Production"

- Tambahkan detail seperti deskripsi, ikon, screenshot, rating konten, dll.

8.3.2 iOS

Langkah-langkah:

1. Buka Project di Xcode

   open ios/Runner.xcworkspace

2. Setup Apple Developer
   - Tambahkan akun Apple Developer di Xcode > Preferences > Accounts
   - Tambahkan Bundle Identifier di ios/Runner/Info.plist
   - Aktifkan Signing & Capabilities
3. Build dan Archive

   flutter build ios --release
   - Lanjutkan dengan Product > Archive di Xcode
   - Upload ke App Store Connect

8.3.3 Web

Build Web

flutter build web

Hasilnya akan berada di:

build/web/

2. Hosting

Beberapa opsi hosting gratis untuk Web Flutter:

- Firebase Hosting
- Netlify
- Vercel
- GitHub Pages

Contoh: Firebase Hosting

bash

CopyEdit

npm install -g firebase-tools

firebase login

firebase init

firebase deploy

Pastikan memilih build/web sebagai direktori publik.

8.3.4 Desktop

a. Aktifkan Platform Desktop

```
flutter config --enable-windows-desktop
flutter config --enable-macos-desktop
flutter config --enable-linux-desktop
```

b. Build Aplikasi Desktop

```
flutter build windows
flutter build macos
flutter build linux
```

File hasil build akan berada di:

```
build/windows/runner/Release/
build/macos/Build/Products/Release/
build/linux/...
```

Anda bisa membuat installer sendiri (contoh: `Inno Setup` untuk Windows).

8.4 Tips Deployment

Agar proses deployment berjalan lancar dan menghasilkan aplikasi yang optimal, berikut beberapa tips penting yang perlu diperhatikan:

- Minify Code

  Selalu gunakan mode --release saat membangun aplikasi untuk menghasilkan kode yang lebih kecil dan lebih cepat.
- Uji Aplikasi di Perangkat Asli

  Pastikan aplikasi diuji langsung pada perangkat fisik (Android/iOS), bukan hanya emulator, untuk mengetahui performa dan kompatibilitas nyata.
- Optimasi Ukuran Aplikasi

  Gunakan fitur tree shaking (secara otomatis aktif di release mode) untuk menghapus kode yang tidak digunakan, dan pastikan asset yang tidak diperlukan sudah dihapus dari project.
- Dokumentasi Aplikasi

  Siapkan deskripsi aplikasi, screenshot, dan video demo untuk keperluan pengunggahan ke Play Store, App Store, atau web.

8.5 Ringkasan Perintah Penting

Tabel 8. 2 Ringkasan Perintah

Platform	Perintah Build
Android APK	flutter build apk --release
Android AAB	flutter build appbundle --release
iOS	flutter build ios --release
Web	flutter build web
Desktop (Windows)	flutter build windows
Desktop (macOS)	flutter build macos
Desktop (Linux)	flutter build linux